

Big Data

Chapter 5

Classification of the NoSQL databases

Jorge Bernardino
jorge@isec.pt


SQL walks into a bar.
It says to the bartender, 'Hey, I have a big group coming. Will we all have a place to sit?'
And the bartender says 'No. But if you want, you can join the tables!'
Good one, Soupy.
© 2017 The Code Zone
Visit angriestprogrammer.com for your dose of cynicism

Agenda

- Key-Value
- Column
- Document
- Graph

NoSQL



NoSQL is a non-relational database management system, which does not need to have a fixed schema.



NoSQL database system contains various database technologies that can manage structured, unstructured, and semi-structured.



We must use the relevant database to solve a specific set of problems as there are various solutions that are better than all of the others.

Types of NoSQL Databases

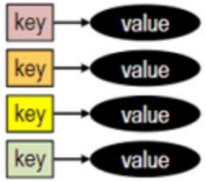
NoSQL Databases are mainly categorized into four types:

- Key-value
- Column-oriented
- Document-oriented
- Graph-based

NoSQL Databases

Type	Example
Key-Value Store	 
Wide Column Store	 
Document Store	 
Graph Store	 

Key Value



Unlike traditional relational databases with rows and columns, key-value NoSQL databases offer a simpler approach: storing data as key-value pairs. This makes them incredibly fast and scalable, particularly for large datasets or scenarios requiring unstructured data.

How key-value NoSQL databases work:

1. Data Organization: Imagine a giant dictionary. Each entry has a unique key (like a word) and an associated value (like the word's definition). Similarly, in a key-value NoSQL database, each piece of data has a unique key that identifies it and a value that stores the actual information.

2. Accessing Data: Retrieving data is straightforward. We simply provide the key, and the database quickly retrieves the corresponding value. It's like looking up a word in a dictionary – fast and efficient.

Key Value

3. Data Types: The beauty lies in the flexibility of values. They can be simple strings, numbers, complex objects, or even entire documents. This versatility makes key-value NoSQL databases suitable for various data types, from user profiles to social media posts.

4. Schema-less or Flexible Schema: Unlike relational databases with predefined schemas (structures), key-value NoSQL databases can be schema-less or have flexible schemas. This means we don't need to rigidly define data structures beforehand, allowing for more agile development and adaptability to changing data needs.

Key Value

Benefits of Key-Value NoSQL Databases:

- High performance and scalability: Their simple structure enables faster reads and writes, making them ideal for handling massive datasets and high-traffic applications.
- Flexibility and agility: The lack of rigid schemas allows for easy adaptation to evolving data models and diverse data types.
- Horizontal scalability: Adding more servers easily scales the database, unlike vertically scaling traditional databases with limited capacity.
- Simplicity and ease of use: Their straightforward key-value structure makes them easier to learn and manage compared to complex relational databases.

Key Value

Challenges of Key-Value NoSQL Databases:

- Limited querying capabilities: While fast for retrieving single values, complex queries across multiple key-value pairs can be challenging.
- Data consistency complexities: Maintaining data consistency across multiple distributed servers requires additional considerations.
- Potential lack of data relationships: Understanding relationships between data pieces might require additional effort due to the absence of predefined table structures.

Key Value

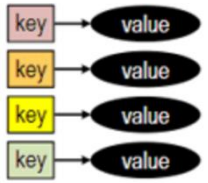
Use Cases for Key-Value NoSQL Databases:

- Caching frequently accessed data: Storing user profiles, shopping cart items, or session data for faster retrieval.
- Real-time applications: Handling high-volume data streams and enabling real-time data analysis.
- IoT and sensor data storage: Efficiently storing and accessing sensor readings or device information.
- Large-scale content management: Managing large collections of unstructured data like images, videos, or social media posts.

Overall, key-value NoSQL databases offer a compelling alternative for high-performance, scalable data storage, especially for unstructured or frequently accessed data.

However, understanding their limitations and choosing the right use case is critical to successfully realizing their potential.

Key Value



Based on distributed hash tables and Amazon's Dynamo paper.

Created to handle massive amounts of data and store schema-free data.

Data insertion and retrieval are simple.

Stores data as a hash table where all keys are unique and the value can be a string, JSON, BLOB, etc.

Values can be stored against strings, lists, hashes sets and sorted sets as a key.

Collections, associative arrays, and dictionaries, etc. can be used as key-value databases.

Key Value

How it works

- **Data Storage and Retrieval**
- When data is stored, the **key** is used as an index to quickly locate and retrieve the corresponding value.
- A request like *GET user:1001* retrieves the value **without scanning the entire database**.

Example

```
db.set("user:1001", {"name": "Alice", "age": 25})
db.get("user:1001")  # Returns: {"name": "Alice", "age": 25}
```

Key Value

Basic Structure:

A key-value database stores data as **key-value pairs** in a simple and efficient format:

Key → *Value*

"user:1001" → *{"name": "Alice", "age": 25}*

"product:5002" → *{"title": "Smartphone", "price": 699}*

- **Key:** A unique identifier for the data (e.g., a string or UUID - Universally Unique Identifier).
- **Value:** The associated data, which can be **a string, JSON object, binary file, or even a blob of data**.

Key Value

- Data is stored in key/value pairs.
- It is designed in such a way to handle lots of data and heavy load.
- Key-value pair storage databases store data as a hash table where each key is unique, and the value can be a JSON, BLOB (Binary Large Objects), string, etc.

- Example: a key-value pair may contain a key like "Website" associated with a value like ”ISEC".

Key Value

Key	Value
Name	Joe Bloggs
Age	42
Occupation	Stunt Double
Height	175cm
Weight	77kg

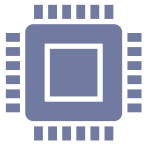
Key Value



It is one of the most basic NoSQL database example.



This kind of NoSQL database is used as a collection, dictionaries, associative arrays, etc.



Key value databases help the developer to store schema-less data.



They work best for shopping cart contents.



Redis, Memcached, Hazelcast, Riak are some NoSQL examples.

Key Value Databases



A key-value store is a simple hash table, primarily used when all access to the database is via primary key.



Think of a table in a traditional RDBMS with two columns, such as ID and NAME, the ID column being the key and NAME column storing the value.



In an RDBMS, the NAME column is restricted to storing data of type String. The application can provide an ID and VALUE and persist the pair; if the ID already exists the current value is overwritten, otherwise a new entry is created.

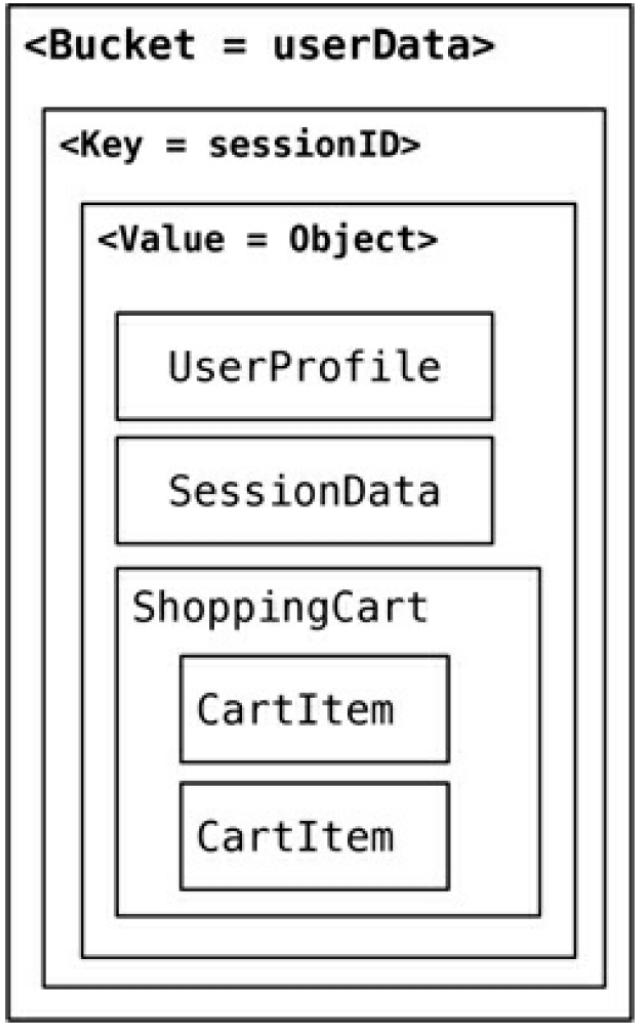
Key Value Databases

- Let's look at how terminology compares in Oracle and Riak:

Oracle	Riak
database instance	Riak cluster
table	bucket
row	key-value
rowid	key

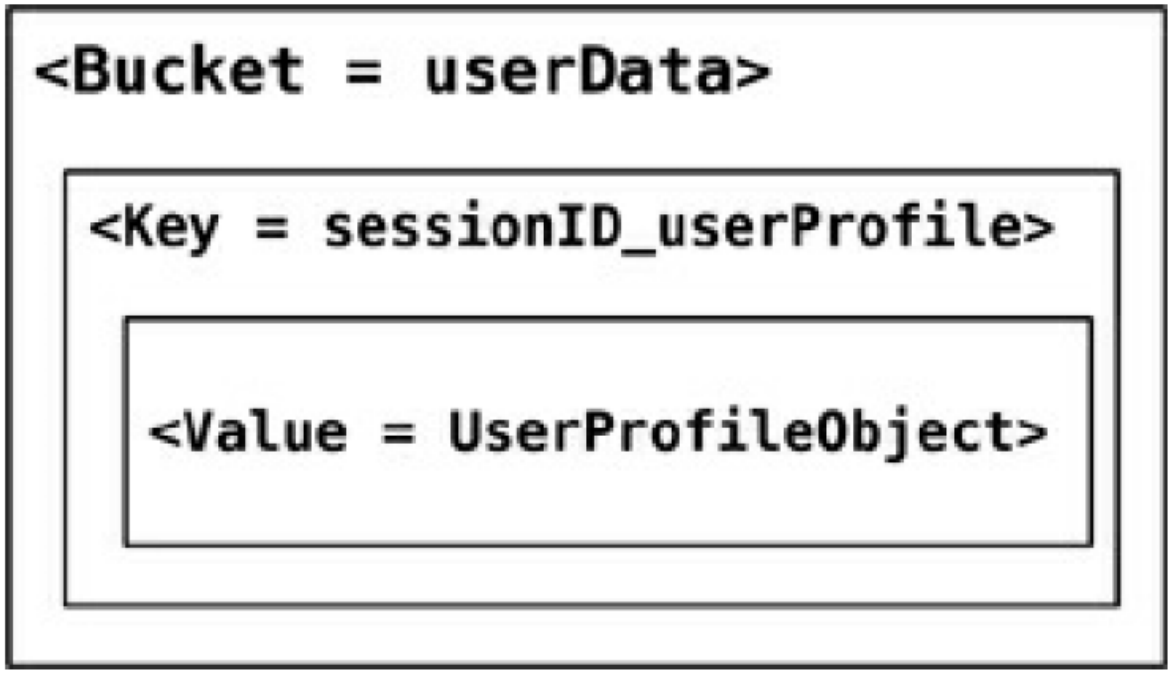
What is a Key-Value Store ?

- Storing all the data in a single bucket:



What is a Key-Value Store ?

- Change the key design to segment the data in a single bucket:



Redis CRUD operations



Redis CRUD operations

- The Redis command-line interface (CLI) is a more common approach for developers to interact with data from a Redis database

- The basic command to get a key-value pair is to call **get** with the key:

```
localhost:6379> get key:1
"value1"
```

Redis CRUD operations

- Setting a value can be done with the **set** command:

```
localhost:6379> set key:1 value1new
OK
localhost:6379> get key:1
"value1new”
```

- Creating a value can also be done in the same way:

```
localhost:6379> get key:3
(nil)
localhost:6379> set key:3 value3new
OK
localhost:6379> get key:3
"value3new"
```

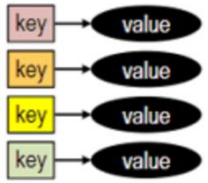
Redis CRUD operations

- And deletion can be done with the **del** command:

```
localhost:6379> del key:3
(integer) 1
localhost:6379> get key:3
(nil)
localhost:6379>
```

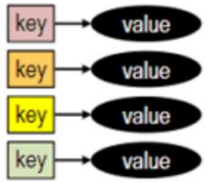
- This is just the basics of the Redis CLI.
- We can do more advanced operations through the CLI.
- An extensive list of commands can be found here:
  https://redis.io/commands/

Key Value: Use cases



- Key-value databases are ideal for storing session information, user profiles, comments, and recommendations of products.
- **Redis**:
  - **Pinterest** uses to store lists of images, galleries, users, followers, and un-followers.
  - **Coinbase** uses to authorize rate points and assurance the rightness of the Bitcoin transactions.
  - **Twitter** uses to manage the Twitter timeline.
- **Memcached**:
  - **Quora** uses to maintain the cache of somewhat defined databases.

Key Value



Popular Key-Value NoSQL Database

Database	Features
Redis	In-memory, fast, supports lists, sets, and expiration policies.
Amazon DynamoDB	Fully managed, highly scalable cloud database.
Riak	Distributed, fault-tolerant key-value store.
Memcached	High-performance, distributed memory object caching system.

Column-oriented



Column NoSQL databases, also known as wide-column stores, offer a unique data storage approach different from both relational and document-oriented NoSQL databases.

How Column NoSQL databases work?

Data organization

- **Columns**: Unlike rows and columns in relational databases or documents in document-oriented stores, data is organized into columns or column families. Each column stores a specific type of data, like username, email, or purchase history.
- **Rows**: Within each column, data is further organized into rows identified by a key (like a user ID). Each row has values only for the existing columns, allowing for flexibility in data structure.
- **Dynamic schema**: Column families can be added or removed without affecting existing data, providing a dynamic schema that adapts to changing data needs.

Column-oriented



How Column NoSQL databases work?

Data Access

- **Column-centric queries**: Queries primarily focus on retrieving specific columns or groups of columns across multiple rows. This is efficient for scenarios like filtering users by location or analyzing purchase history by product category.
- **Row-level access**: Row-level access is also possible, but it might be slower due to the lack of a dedicated row structure.

Column-oriented

Benefits :

- High performance and scalability: Column-based organization allows for efficient retrieval of specific data elements, leading to high performance for read and write operations. The database can be easily scaled horizontally by adding more servers.
- Flexible schema: The dynamic schema allows for storing diverse data types and easily adapting to evolving data models without restructuring the entire database.
- Efficient for specific use cases: They excel in scenarios where frequent queries involve specific columns or groups of columns, like analyzing sensor data or user attributes.

Column-oriented

Challenges:

- Complex querying: Performing complex joins or aggregations across multiple columns or rows can be challenging due to their non-relational structure.
- Data consistency considerations: Maintaining data consistency across multiple distributed servers requires careful attention, especially with updates spanning multiple columns.
- Limited row-level access: Retrieving entire rows might not be as efficient as in relational databases, requiring careful query design.

Column-oriented

Use Cases:

- Sensor data storage and analysis: Efficiently storing and analyzing sensor data with varying timestamps and measurements, leveraging column-based organization for specific data access.
- Telecom and financial data: Storing and querying large datasets with diverse attributes like call records, transactions, or customer information, often requiring access to specific columns.
- Log data analysis: Analyzing large log files with diverse data points, where efficient access to specific log fields is crucial.
- Recommendation systems: Storing and analyzing user preferences and product attributes in columns for personalized recommendations based on specific characteristics.

Overall, column NoSQL databases offer a powerful solution for scenarios requiring efficient access to specific data elements and flexible schema management. However, understanding their limitations and choosing the right use case is crucial for maximizing their benefits and ensuring efficient data access and analysis.

Column-oriented



isec Engenharia

These databases emerged after Google's research on the Google file system named BigTable distributed storage system.

Column-oriented databases basically designed to work on columns, every column is treated separately.

All values are stored contiguously for each column.

Column-oriented databases stores data in column specific files with the same type.

Column-oriented



Storage Format: Column vs. Row

- **Row-based databases** (e.g., MySQL, PostgreSQL) store data **row by row**. Each row contains all its fields together.
- **Columnar databases** (e.g., Apache Cassandra, Apache Hbase, Apache Accumulo, Amazon Keyspaces) store **each column separately** instead of keeping complete rows together.

Example:

ID	Name	Age	Salary
1	Alice	25	50000
2	Bob	30	60000
3	Carol	35	70000

Column-oriented



Row-based storage (traditional databases like MySQL, PostgreSQL):

1, Alice, 25, 50000

2, Bob, 30, 60000

3, Carol, 35, 70000

Column-based storage:

ID: 1, 2, 3

Name: Alice, Bob, Carol

Age: 25, 30, 35

Salary: 50000, 60000, 70000

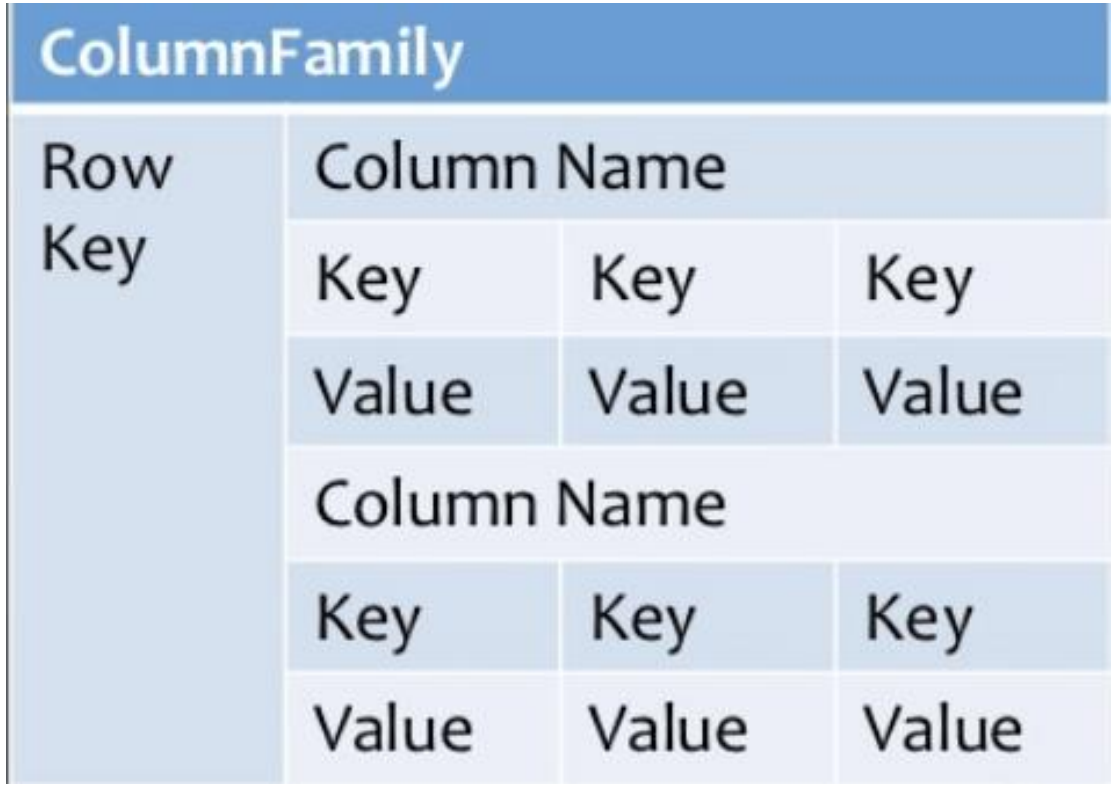
Column-based

Column-oriented databases work on columns and are based on BigTable paper by Google.

Every column is treated separately.

Values of single column databases are stored contiguously.

Column-based



Example (Cassandra)

In a system like **Apache Cassandra**, a column family could be:

```
CREATE TABLE users (
    user_id UUID PRIMARY KEY,
    name TEXT,
    email TEXT,
    phone_numbers LIST<TEXT>
);
```

Here, users is a column family. For each user, you can have a **user_id** (which uniquely identifies the row) and columns such as name, email, and phone_numbers.

Some users might have different columns depending on the data they have, making this schema flexible.

Column-based



They deliver high performance on aggregation queries like SUM, COUNT, AVG, MIN etc. as the data is readily available in a column.



Column-based NoSQL databases are widely used to manage Data Warehouses, Business Intelligence, CRM, Library card catalogs.



Examples: Cassandra, HBase, Accumulo, Hypertable, …

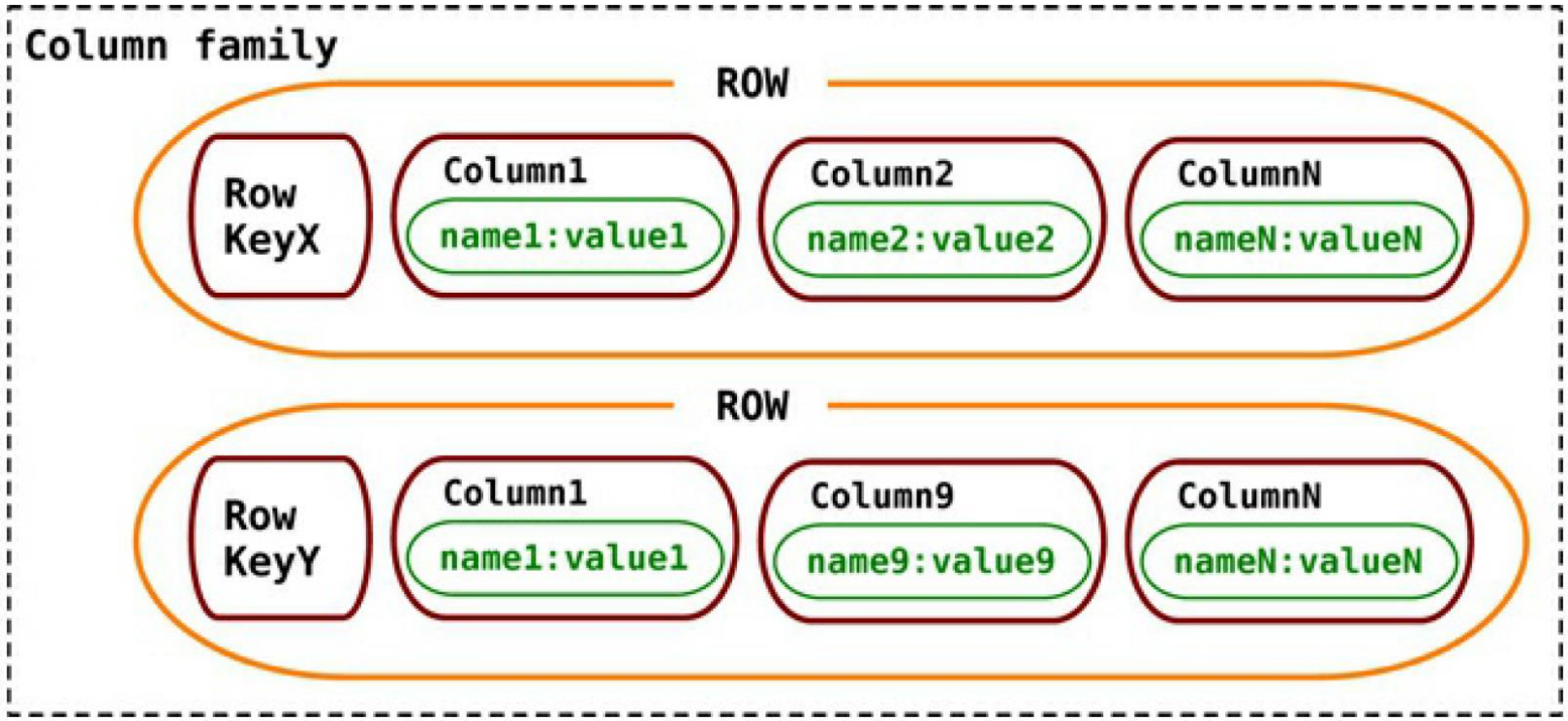
Column-Family

- Column-family databases, such as Cassandra and HBase, allow to store data with keys mapped to values and the values grouped into multiple column families, each column family being a map of data.

RDBMS	Cassandra
database instance	cluster
database	keyspace
table	column family
row	row
column (same for all rows)	column (can be different per row)

What is a Column-Family Database?

Cassandra's data model with column families



Cassandra can be described as fast and easily scalable with write operations spread across the cluster. The cluster does not have a master node, so any read and write can be handled by any node in the cluster.

Cassandra Replication and Fault Tolerance

- Cassandra uses replication to store multiple copies of data across nodes.

Example

```
CREATE KEYSPACE mykeyspace
WITH replication = {'class': 'NetworkTopologyStrategy', 'datacenter1': 3};
```

- This means data is replicated **3 times** in datacenter1.
- If a node goes down, the system remains operational.

Cassandra

Key Benefits of No Master Node:

- ✓ No single point of failure
- ✓ High availability (even if some nodes fail)
- ✓ Scalability (add more nodes anytime)

How is data structured in Cassandra?

- The column has a key of Name and the value of Martin and has a timestamp attached to it.

```
{
firstname: "fullName",
value: "Martin Fowler",
timestamp: 12345667890
}
```

How is data structured in Cassandra?

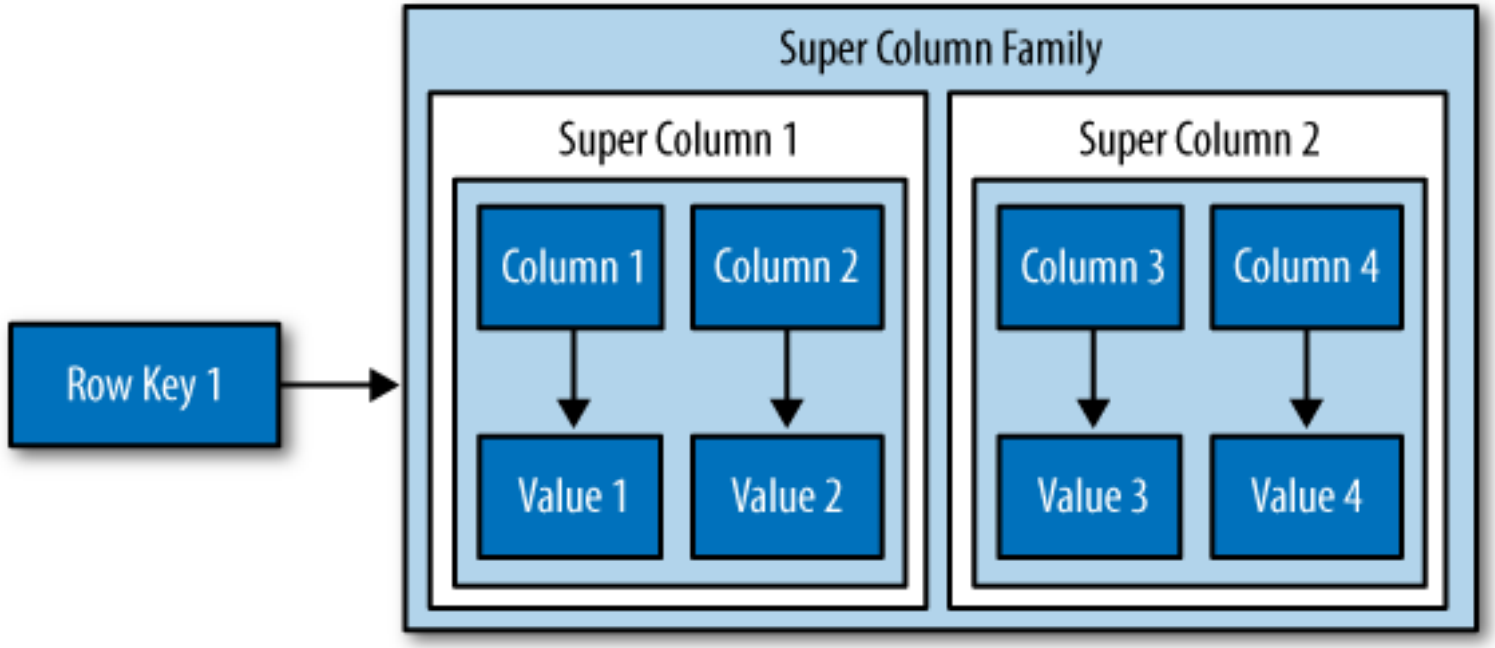
- A row is a collection of columns attached or linked to a key; a collection of similar rows makes a column family.
- When the columns in a column family are simple columns, the column family is known as **standard column family.**

```
//column family
{ //row
”antonio-costa" : {
firstName: ”Antonio",
lastName: ”Costa",
lastVisit: "2025/03/06"
}
//row
”joao-silva" : {
firstName: ”Joao",
lastName: ”Silva",
location: ”Coimbra"
}
}
```

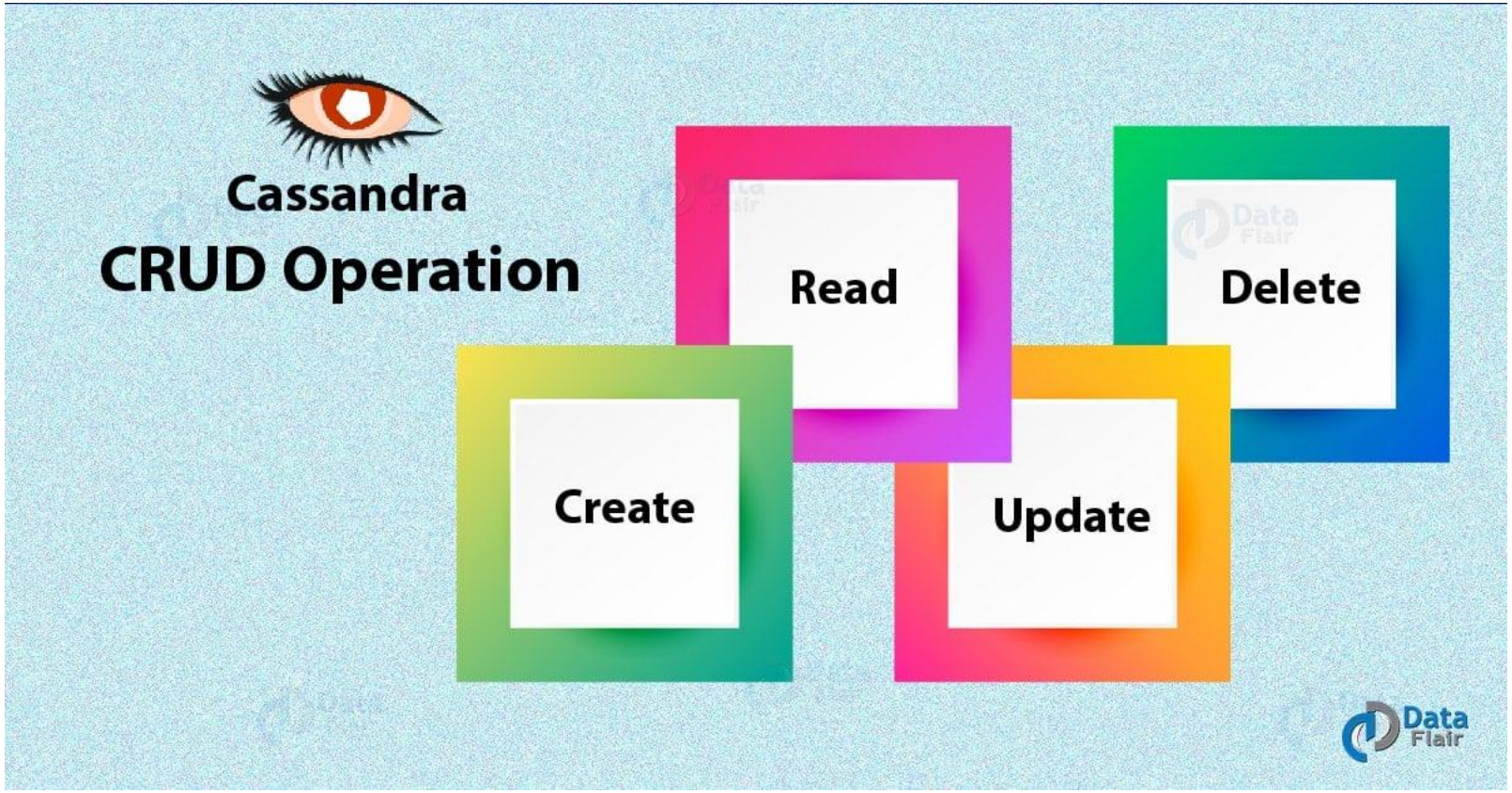
Each column family can be compared to a container of rows in an RDBMS table where the key identifies the row and the row consists on multiple columns.
The difference is that various rows do not have to have the same columns, and columns can be added to any row at any time without having to add it to other rows.
We have the *antonio-costa* row and the *joao-silva* row with different columns; Both rows are part of the column family.

How is data structured in Cassandra?

- When a column consists of a map of columns, then we have a **super column**.
- A super column consists of a name and a value which is a map of columns.
- Think of a super column as a container of columns.



Cassandra CRUD operations



Cassandra Create operation

- A user can insert data into the table using Cassandra CRUD operation. The data is stored in the columns of a row in the table. Using INSERT command with proper what, a user can perform this operation.

- Syntax of **Create** Operation:

```
INSERT INTO <table name>
(<column1>,<column2>....)
VALUES (<value1>,<value2>...)
USING<option>
```

Cassandra Create operation

- Creating a table and inserting the data into a table.
- A table with information about students in college.

```
cqlsh:keyspace1> INSERT INTO student(en, name, branch, phone, city)
VALUES(001, 'Ayush', 'Electrical Engineering', 9999999999, 'Boston');

cqlsh:keyspace1> INSERT INTO student(en, name, branch, phone, city)
VALUES(002, 'Aarav', 'Computer Engineering', 8888888888, 'New York City');

cqlsh:keyspace1> INSERT INTO student(en, name, branch, phone, city)
VALUES(003, 'Kabir', 'Applied Physics', 7777777777, 'Philadelphia');
```

EN	NAME	BRANCH	PHONE	CITY
001	Ayush	Electrical Engineering	9999999999	Boston
002	Aarav	Computer Engineering	8888888888	New York City
003	Kabir	Applied Physics	7777777777	Philadelphia

Cassandra Update operation

- Uses two keywords while updating the table.
  - Where: Specify the location where data is to be updated.
  - Set: Specify the updated value.
- If the row **does not exist**, Cassandra **creates it automatically** (this behavior is called **"Upsert"**).

- Syntax of **Update** Operation:

```
UPDATE <table name>
SET <column name>=<new value>
<column name>=<value>...
WHERE <condition>
```

Cassandra Update operation

- In the example, we will update Aarav's city from 'New York City' to 'San Fransisco'.
- INPUT:

```
cqlsh:keyspace1> UPDATE student SET city='San Fransisco'
WHERE en=002;
```

EN	NAME	BRANCH	PHONE	CITY
001	Ayush	Electrical Engineering	9999999999	Boston
002	Aarav	Computer Engineering	8888888888	New York City
003	Kabir	Applied Physics	7777777777	Philadelphia

EN	NAME	BRANCH	PHONE	CITY
001	Ayush	Electrical Engineering	9999999999	Boston
002	Aarav	Computer Engineering	8888888888	San Fransisco
003	Kabir	Applied Physics	7777777777	Philadelphia

Cassandra Read operation

- A user has a choice to read either the whole table or a single column.
- To read data from a table, a user can use SELECT clause.
- This command is also used for verifying the table after every operation.

- SYNTAX to **read** the whole table:

```
SELECT * FROM <table name>;
```

- SYNTAX to **read** selected columns:

```
SELECT <column name1>,<column name2>.... FROM <table name>;
```

Cassandra Read operation

- To read columns of name and city from table ‘student’.

- INPUT:

```
cqlsh:keyspace1> SELECT name, city FROM student;
```

EN	NAME	BRANCH	PHONE	CITY
001	Ayush	Electrical Engineering	9999999999	Boston
002	Aarav	Computer Engineering	8888888888	San Fransisco
003	Kabir	Applied Physics	7777777777	Philadelphia

NAME	CITY
Ayush	Boston
Aarav	San Fransisco
Kabir	Philadelphia

Cassandra Delete operation

- Allows a user to delete data from a table.

- Syntax of **Delete** Operation:

```
DELETE <identifier> FROM <table name> WHERE <condition>;
```

- In the 'student' table let us delete the 'phone' or phone number from 003 row.

```
cqlsh:keyspace1> DELETE phone FROM student WHERE en=003;
```

EN	NAME	BRANCH	PHONE	CITY
001	Ayush	Electrical Engineering	9999999999	Boston
002	Aarav	Computer Engineering	8888888888	San Fransisco
003	Kabir	Applied Physics	null	Philadelphia

Cassandra Delete operation

- SYNTAX for deleting the entire row:

```
DELETE FROM <identifier> WHERE <condition>;
```

- In the 'student' table, let us delete the entire third row.

```
cqlsh:keyspace1> DELETE FROM student WHERE en=003;
```

EN	NAME	BRANCH	PHONE	CITY
001	Ayush	Electrical Engineering	9999999999	Boston
002	Aarav	Computer Engineering	8888888888	San Fransisco
003	Kabir	Applied Physics	null	Philadelphia

EN	NAME	BRANCH	PHONE	CITY
001	Ayush	Electrical Engineering	9999999999	Boston
002	Aarav	Computer Engineering	8888888888	San Fransisco

Column-oriented: Use Cases

The column-oriented database gives a highly scalable architecture and high performance as they are fast for insertion and retrieval.

They have been very popular among organizations working on Internet of Things, Big Data, and personalized content recommendations.

- **Cassandra**:
  - **Spotify** uses to store metadata about artists, songs, etc. and user profile parameters for better personalization.
  - **Outbrain** uses to serve personalized content recommendations.

- **HBase**:
  - **Facebook** used for various services like search indexing and the "Nearby Friends" feature.

When to use Column-oriented?

- ✅ You need **fast writes and scalable storage** for big data.
- ✅ Your queries focus on **specific columns** rather than entire rows.
- ✅ You require **high availability** with a distributed architecture.
- ✅ Your workload includes **time-series, analytics, or IoT data**

Document



In the realm of NoSQL databases, document databases, also known as document-oriented databases, present a flexible and scalable approach to data storage.

How Document NoSQL databases work?

Data organization

- **Documents**: The core element of this database type are documents, essentially JSON-like structures encompassing various fields and their corresponding values. Think of them as self-contained units of information, resembling web pages or application settings.
- **Schema-less or Flexible Schema**: Unlike their relational counterparts with rigid table structures, document databases embrace flexibility. Documents can have different structures and fields without strict pre-definition, allowing adaptation to evolving data needs.

Document



How Document NoSQL databases work?

Data Access

- **Document-centric Queries**: Retrieving entire documents based on specific criteria applied to various fields within them is the primary focus. Imagine searching for a book in a library by its title or author – similar principles apply here.

- **Indexing and Filtering**: While primarily document-centric, queries can also filter and retrieve specific pieces of data (fields) across multiple documents through efficient indexing mechanisms.

Document

Benefits :

- Flexibility and Agility: The schema-less approach and document structure enable storing diverse data types and adapting to changing data models, fostering agile development.
- Performance and Scalability: Like other NoSQL databases, document databases offer high performance for reads and writes and can scale horizontally by adding more servers.
- Rich Data Representation: Documents can accommodate complex data structures, making them suitable for scenarios requiring nested information or diverse attributes within a single unit.

Document

Challenges:

- Complex Querying: While offering document-centric and limited field-level querying, they might not be ideal for intricate joins or aggregations across large datasets like traditional relational databases.
- Data Consistency Considerations: Maintaining data consistency across distributed servers demands careful attention, especially when updates involve multiple documents.
- Data Management Tools: Choosing the right tools for managing and querying document databases is crucial for efficient data access and manipulation.

Document

Use Cases:

- E-commerce product catalogs: Storing product information with rich details, specifications, and reviews within flexible document structures.
- Content Management Systems (CMS): Managing diverse content types like articles, blog posts, or user profiles with flexibility for different data attributes.
- Internet of Things (IoT) data storage: Efficiently storing and querying sensor data with varying formats and timestamps while leveraging document-level organization.
- Real-time analytics: Enabling real-time processing and analysis of data streams due to their high performance and scalability.

Overall, document NoSQL databases offer a compelling option for applications demanding flexibility, performance, and rich data representation in their storage needs. Understanding their strengths and limitations along with your specific use case is key to making informed decisions and unleashing their full potential.

Document-Oriented



It is a collection of documents, and a document is a collection of key-value type. All values or the data stored in documents can be accessed using the keys.

All documents are independent of each other.

Documents are schema-free and therefore are flexible and easy to modify.

Collections are used to store documents in order to group different kinds of data.

Documents may have nested documents, various different key-value pairs, or key-array pairs.

Document-Oriented



Col1	Col2	Col3	Col4
Data	Data	Data	Data
Data	Data	Data	Data
Data	Data	Data	Data

Document 1

```
{
  "prop1": data,
  "prop2": data,
  "prop3": data,
  "prop4": data
}
```

Document 2

```
{
  "prop1": data,
  "prop2": data,
  "prop3": data,
  "prop4": data
}
```

Document 3

```
{
  "prop1": data,
  "prop2": data,
  "prop3": data,
  "prop4": data
}
```

In this diagram on the left we have rows and columns, and in the right, we have a document database which has a similar structure to JSON.
In the relational database, we have to know what columns you have and so on.
For a document database, we have data store like JSON object.
We don't need to define which ones, which makes them more flexible.

Document-Oriented





The document type is mostly used for CMS systems, blogging platforms, real-time analytics and e-commerce applications.

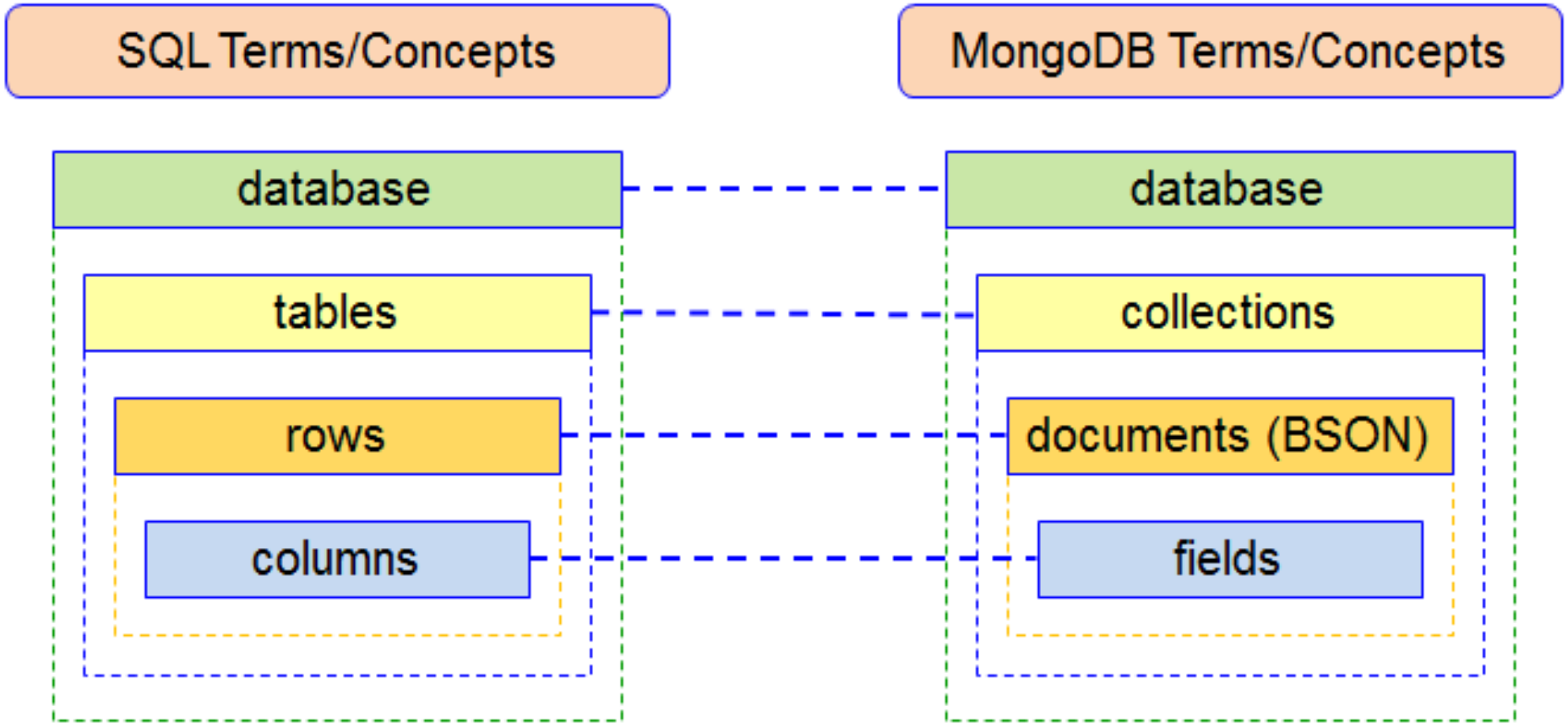


It should not use for complex transactions which require multiple operations or queries against varying aggregate structures.

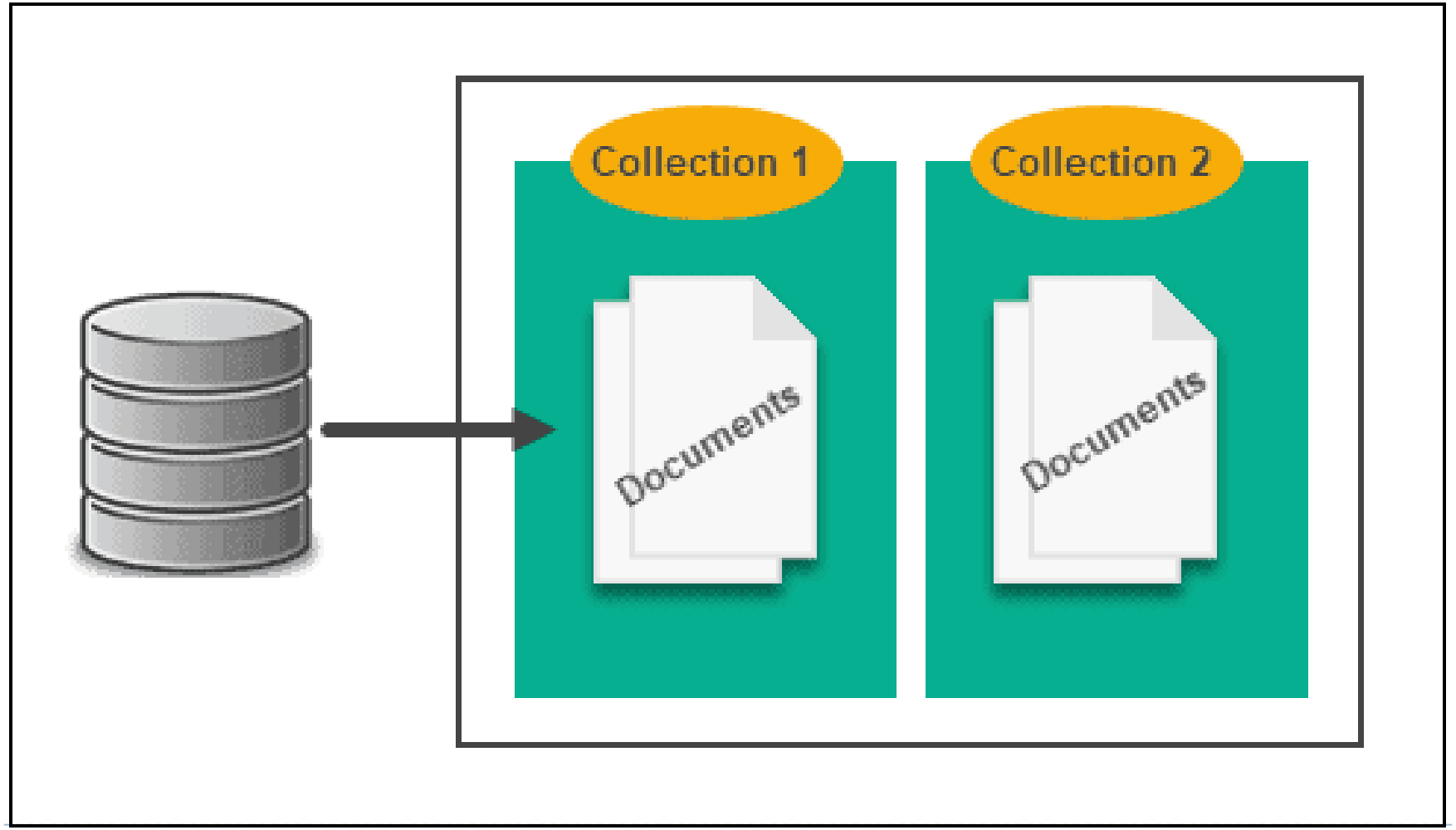


MongoDB and Couchbase are popular Document originated DBMS systems.

Document Databases



Document Databases



Document Databases



- Document databases store documents in the value part of the key-value; think about document databases as key-value databases where the value is searchable. Let's look at how terminology compares in Oracle and MongoDB.

Oracle	MongoDB
database instance	MongoDB instance
schema	database
table	collection
row	document
rowid	_id
join	DBRef

The _id is a special field that is found on all documents in Mongo, just like ROWID in Oracle. In MongoDB, _id can be assigned by the user, as long as it is unique.

What Is a Document Database?

Exemplo 1:

```
{    "firstname": "Martin",
     "likes": [ "Biking",
                "Photography" ],
     "lastcity": "Boston",
     "lastVisited":
}
```

- The above document can be considered a row in a traditional RDBMS.

What Is a Document Database?

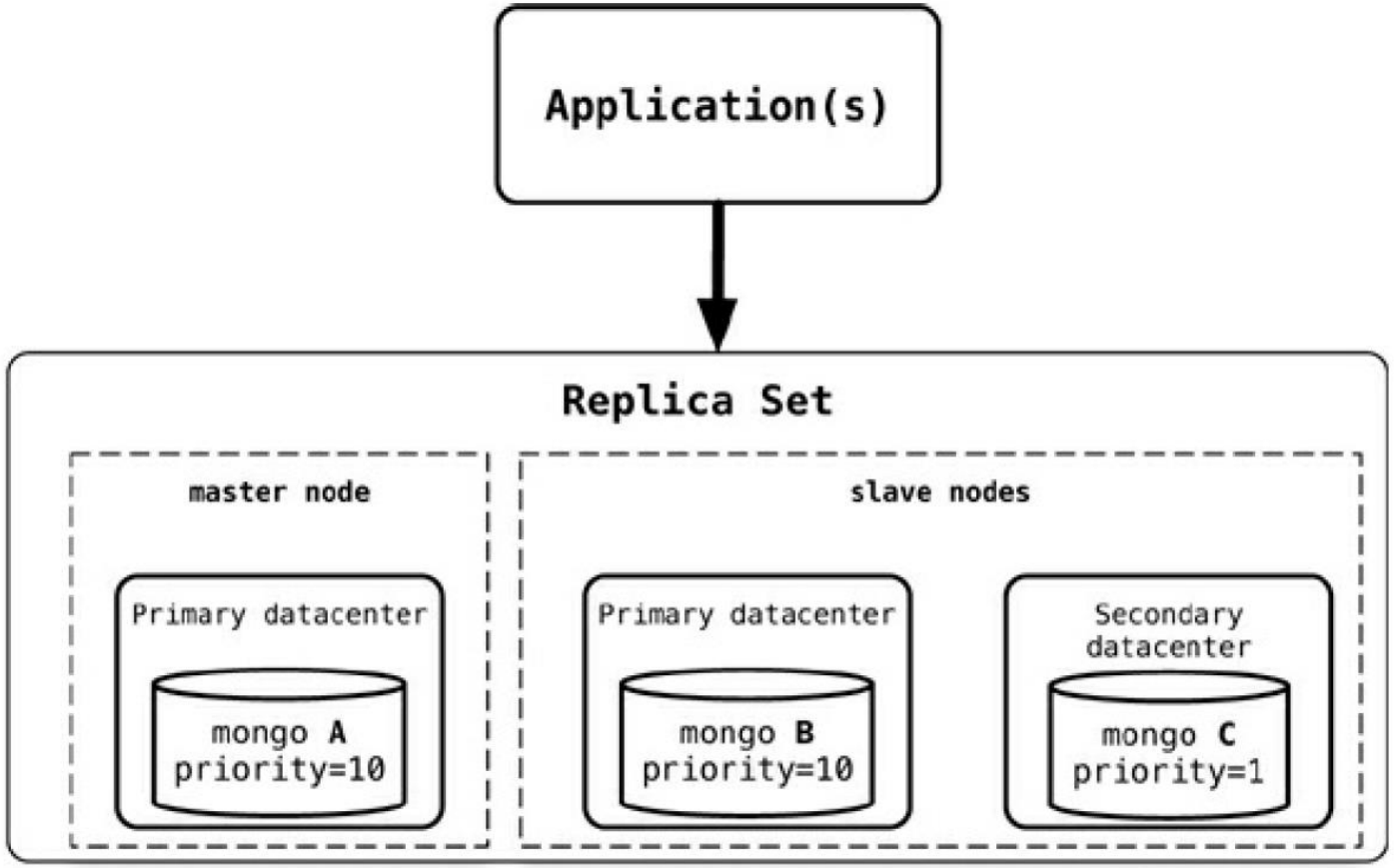
Exemplo 2:

```
{
   "firstname": "Pramod",
   "citiesvisited": [ "Chicago", "London", "Pune", "Bangalore" ],
   "addresses": [
      { "state": "AK",
        "city": "DILLINGHAM",
        "type": "R"
      },
      { "state": "MH",
        "city": "PUNE",
        "type": "R" }
   ],
   "lastcity": "Chicago"
}
```

- Looking at the documents, we can see that they are similar but have differences in attribute names. This is allowed in document databases.

What Is a Document Database?

- Replica set configuration with higher priority assigned to nodes in the same datacenter



MongoDB Queries

- Suppose we want to return all the documents in an order collection (all rows in the Order table). The SQL would be:

  *SELECT * FROM Order*

The equivalent query in Mongo shell would be:

db.order.find()

- Selecting the orders for a single customerId of 883c2c5b4e5b would be:

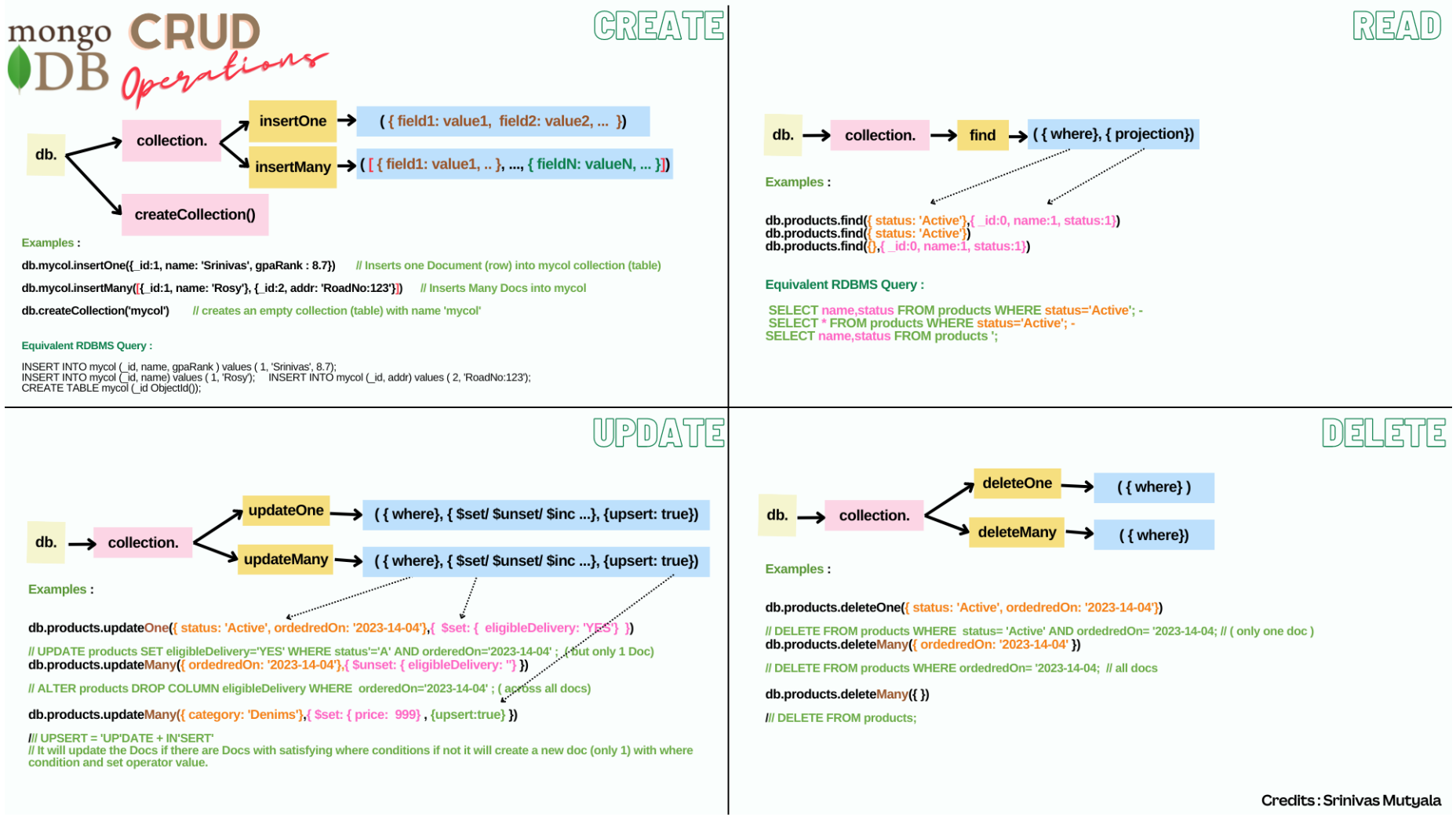
  *SELECT * FROM Order WHERE customerId = "883c2c5b4e5b"*

The equivalent query in MongoDB to get all orders for a single customerId of 883c2c5b4e5b:

db.order.find({customerId: "883c2c5b4e5b"})

MongoDB CRUD Operations

MQL (MongoDB Query Language)

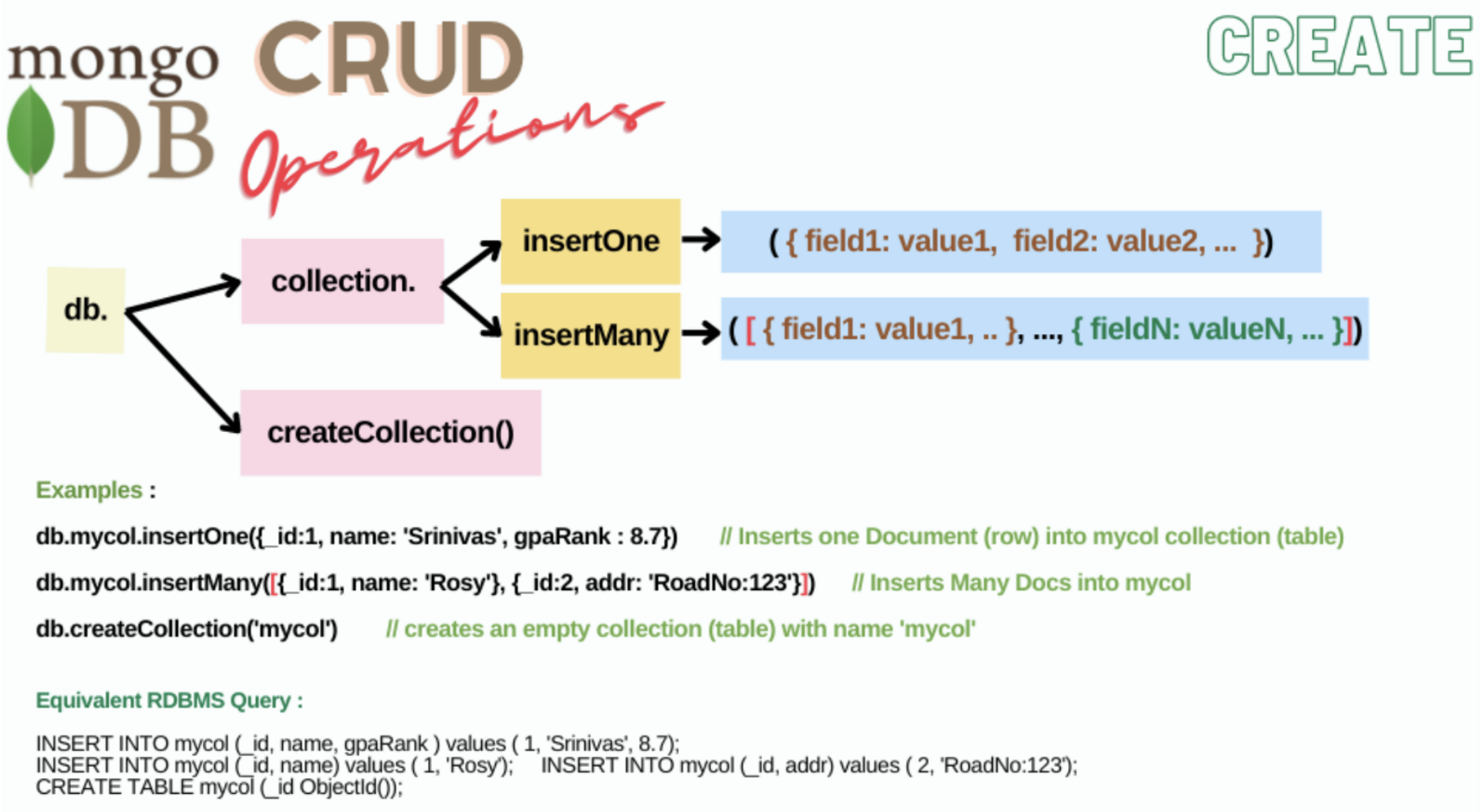


MongoDB Create operation

- Used to insert new data into a MongoDB database.
- In MongoDB, data is organized into collections, which are analogous to tables in a relational database.
- To create a new document in a collection, we can use the insertOne() or insertMany() methods.

- Example of how to insert a new document into a collection called "users":
  - db.users.insertOne({ name: "John Doe", age: 35, email: "johndoe@example.com" })
- The insertOne() method inserts a single document into the "users" collection with the specified fields.

MongoDB Create operation



MongoDB Read operation

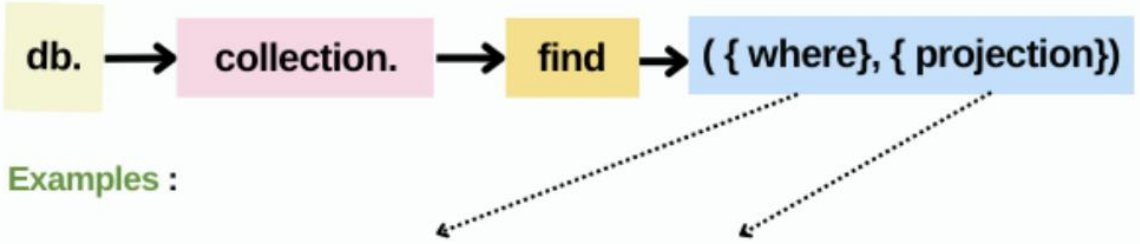
- Used to retrieve data from a MongoDB database.
- To read data from a collection, we can use the find() method.

- Example of how to retrieve all documents from the "users" collection:
  - db.users.find()
- This will return all the documents in the "users" collection.

- We can also specify query criteria to retrieve only specific documents.
- For example, to retrieve all documents where the age is greater than or equal to 30, you can use the following query:
  - db.users.find({ age: { $gte: 30 } })

MongoDB Read operation

READ



Examples :

```
db.products.find({ status: 'Active'},{ _id:0, name:1, status:1})
db.products.find({ status: 'Active'})
db.products.find({},{ _id:0, name:1, status:1})
```

Equivalent RDBMS Query :

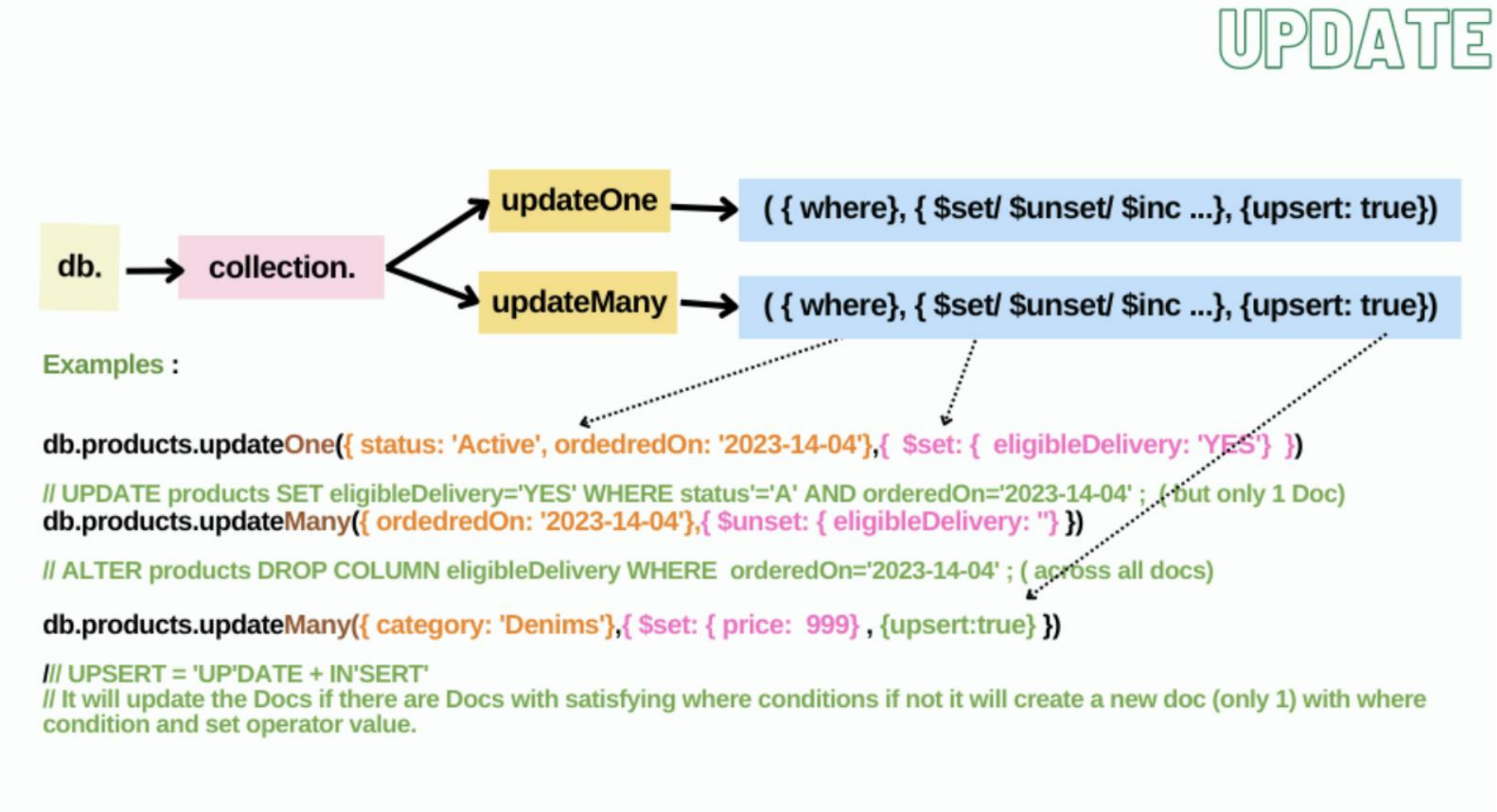
```
SELECT name,status FROM products WHERE status='Active'; -
SELECT * FROM products WHERE status='Active'; -
SELECT name,status FROM products ';
```

MongoDB Update operation

- Used to modify existing data in a MongoDB database.
- To update a document in a collection, we can use the updateOne() or updateMany() methods.
- Example of how to update the email address of a user with the name "John Doe":
  - db.users.updateOne({ name: "John Doe" }, { $set: { email: "newemail@example.com" } })

- The updateOne() method updates the first document that matches the query criteria. The $set operator sets the value of the email field to "newemail@example.com".

MongoDB Update operation



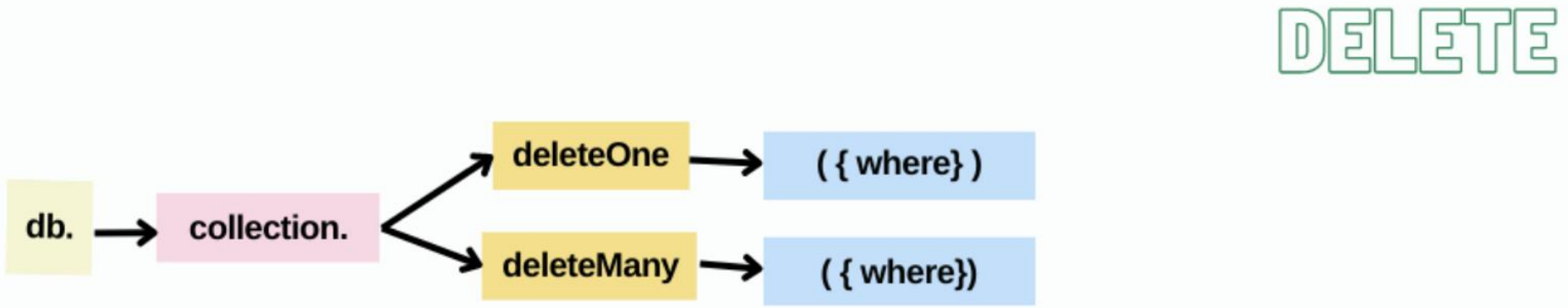
MongoDB Delete operation

- Used to remove data from a MongoDB database.
- To delete a document from a collection, we can use the deleteOne() or deleteMany() methods.

- Example of how to delete a document from the "users" collection where the name is "John Doe":
  - db.users.deleteOne({ name: "John Doe" })

- The deleteOne() method removes the first document that matches the query criteria.

MongoDB Delete operation



Examples :

```
db.products.deleteOne({ status: 'Active', ordedredOn: '2023-14-04'})

// DELETE FROM products WHERE  status= 'Active' AND ordedredOn= '2023-14-04; // ( only one doc )
db.products.deleteMany({ ordedredOn: '2023-14-04' })

// DELETE FROM products WHERE ordedredOn= '2023-14-04;  // all docs

db.products.deleteMany({ })

/// DELETE FROM products;
```

Document-oriented: Use Cases

1 Amazon – E-Commerce & Retail 🛒

✅ Uses **Amazon DynamoDB** (a document + key-value database) to store:

- **Product catalogs** (each product has unique attributes)
- **Customer profiles & preferences**
- **Order tracking**

✅ Also uses **MongoDB** for personalizing recommendations.

Document-oriented: Use Cases

2 Netflix – Streaming & Personalization 🎬

✅ Uses **Cassandra & Couchbase (document + NoSQL DBs)** for:

- **User profiles & viewing history** (flexible, ever-changing)
- **Content metadata** (different formats for movies vs. series)
- **Real-time recommendations**

- Netflix **handles billions of requests per day**, making document databases **ideal for fast reads and writes**.

Document-oriented: Use Cases

3 Facebook (Meta) – Social Media & Messaging 📱

✅ Uses **MongoDB** & **Cassandra** for:

- **Messenger chat storage** (messages stored as JSON-like documents)
- **User activity logs** (tracking likes, comments, shares)
- **Newsfeed personalization**

- Document databases allow Facebook to handle **huge amounts of unstructured social data** efficiently.

Document-oriented: Use Cases

4 Uber – Ride-Hailing & Location Tracking 🚖

✅ Uses **MongoDB & DynamoDB** for:

- **Real-time ride tracking**
- **Driver & rider preferences**
- **Surge pricing calculations**

- Document databases help Uber **scale across multiple regions** with **low-latency requests**.

Graph Databases



Graph NoSQL databases, also known as graph databases, take a unique approach to data storage and management by leveraging graph theory principles. They represent data as a collection of nodes (entities) connected by relationships (edges), forming a web-like structure. This approach excels in capturing complex relationships between data points, making it ideal for specific use cases.

How Graph NoSQL databases work?

Data organization

- **Nodes**: These are the fundamental building blocks, representing real-world entities like people, products, or events. Each node can have properties containing additional information like names, descriptions, or timestamps.
- **Relationships**: These are the connections between nodes, signifying the interactions or associations they share. Relationships are directed (one-way) or undirected (two-way) and can also have properties describing their nature (e.g., "friends with", "purchased").
- **Graph Structure**: The overall arrangement of nodes and relationships forms the graph structure, enabling efficient exploration and analysis of connections between data points.

Graph

How Graph NoSQL databases work?

Data Access

- **Node-centric queries**: Retrieving specific nodes based on their properties is a common operation. Think of finding a person by name or a product by category.
- **Relationship-based queries**: Traversing relationships between nodes is a key strength. You can find all friends of a person, products similar to a purchased item, or events related to a specific topic.
- **Graph algorithms**: Specialized algorithms like shortest path or community detection help analyze large-scale relationships and patterns within the graph.

Graph

Benefits :

- Modeling complex relationships: Graph databases excel at representing and querying intricate relationships between data entities, making them ideal for social networks, recommendation systems, and fraud detection.
- Fast and efficient querying: Navigating relationships is often faster than traditional JOIN operations in relational databases, especially for complex interconnected data.
- Flexible schema: Similar to other NoSQL databases, graph databases are schema-less or have flexible schemas, allowing for adaptability to evolving data models.

Graph

Challenges:

- Learning curve: Understanding graph concepts and query languages requires some initial investment compared to relational databases.
- Limited querying for non-relational data: While powerful for relationships, graph databases might not be optimal for simple queries or aggregations on individual node properties.
- Data consistency considerations: Maintaining data consistency across distributed servers can be more complex due to the interconnected nature of the data.

Graph Databases

Use Cases:

- Social networks: Modeling user connections, interactions, and content sharing based on relationships between users.
- Fraud detection: Identifying suspicious patterns and connections between transactions or entities to uncover potential fraudulent activities.
- Recommendation systems: Analyzing user preferences and product relationships to recommend relevant items or content.
- Knowledge graphs: Building comprehensive knowledge bases with interconnected entities and relationships for various applications like search engines or chatbot assistants.

Overall, graph NoSQL databases offer a powerful tool for storing and analyzing data with complex relationships. Carefully consider their strengths and limitations in relation to your specific use case to determine if they can unlock valuable insights and connections within your data.

Graph



Graph network database stores data in a graph like structure.

Graph databases used for storing frequently changing data.

Graph databases are capable of representing any kind of data. It provides high accessibility as it uses indexes for lookups.

A graph database is a collection of one or more nodes and edges.

Graph



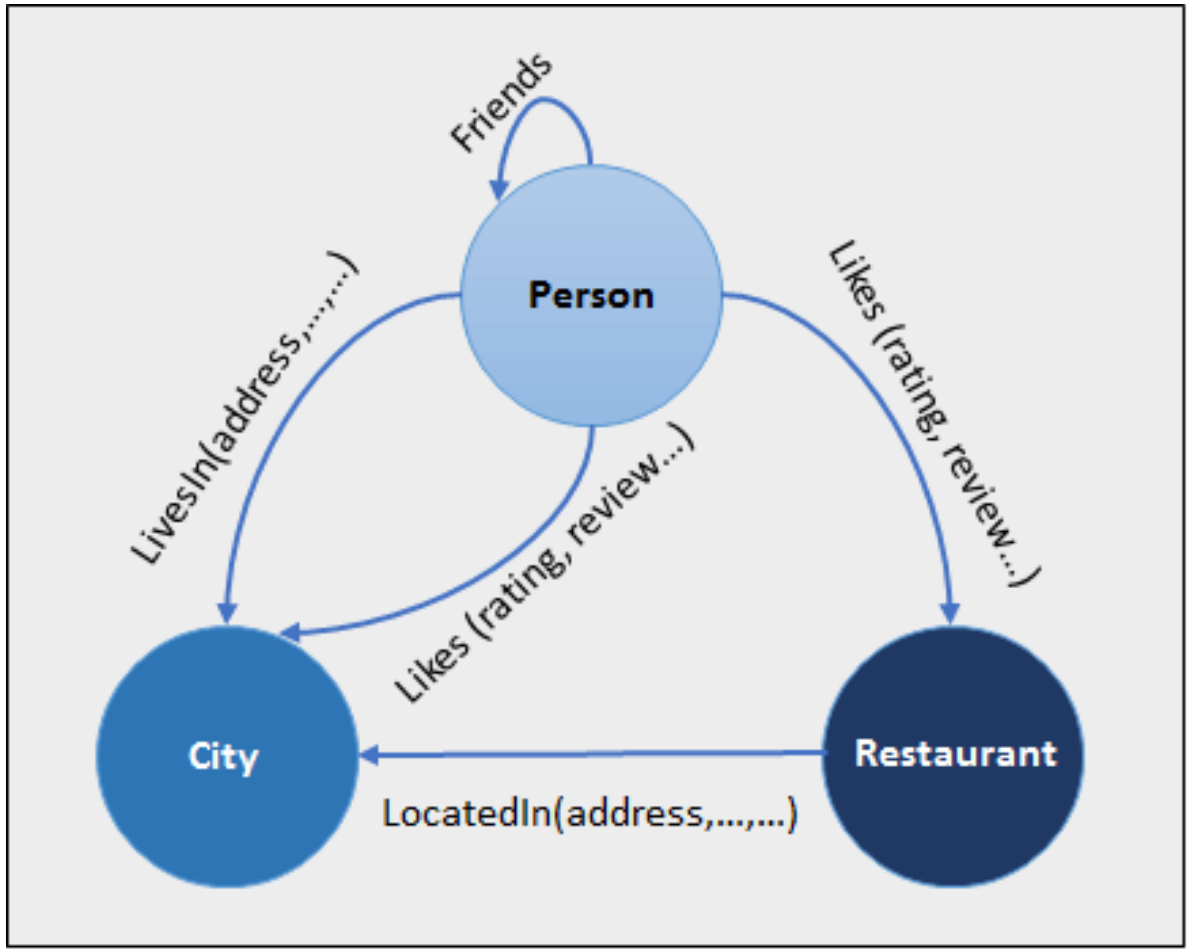
A graph type database stores entities as well the relations amongst those entities.

The entity is stored as a node with the relationship as edges.

An edge gives a relationship between nodes.

Every node and edge has a unique identifier.

Graph


Friends
Person
LivesIn(address,...,...)
Likes (rating, review...)
Likes (rating, review...)
City
Restaurant
LocatedIn(address,...,...)

Graph



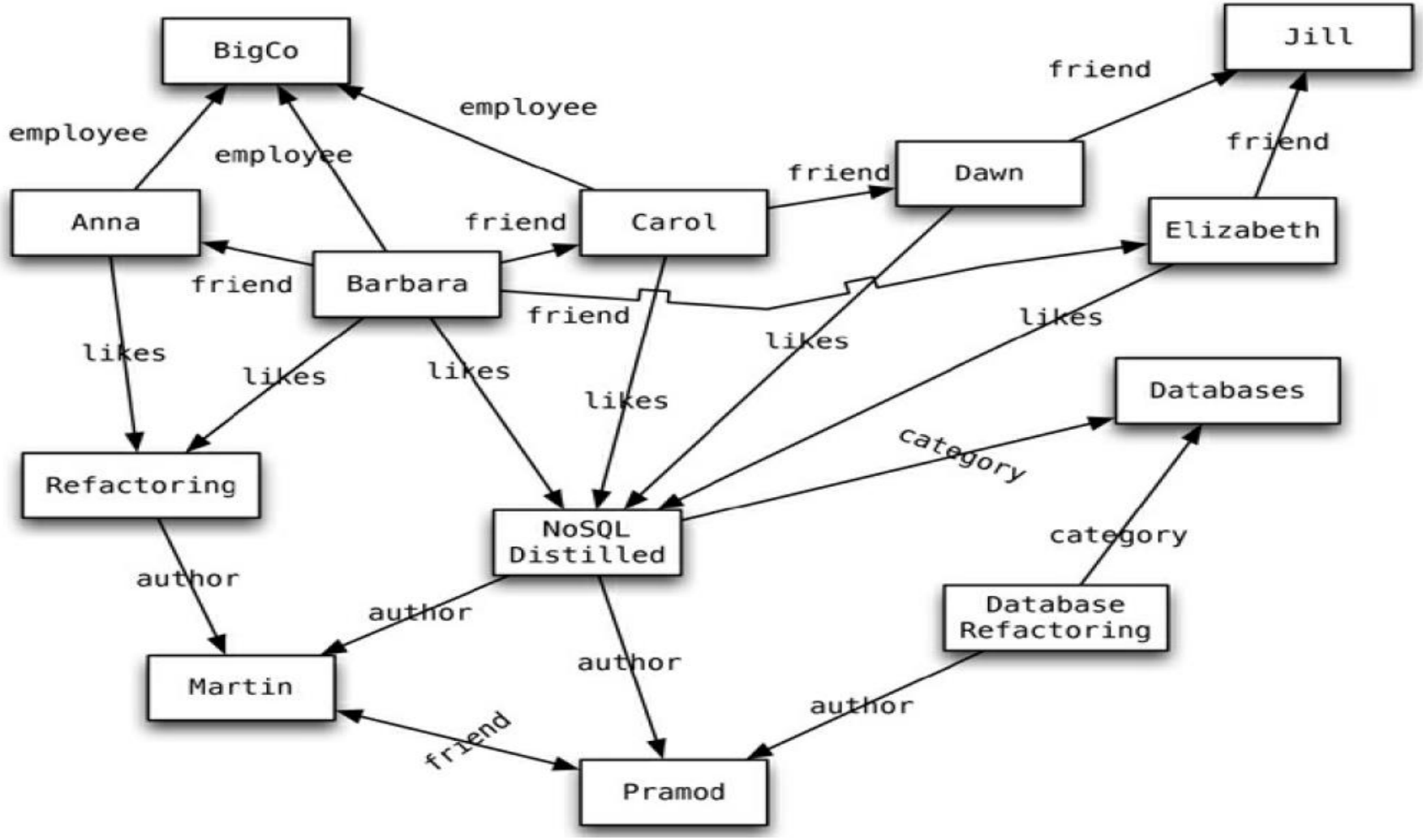
Compared to a relational database where tables are loosely connected, a Graph database is a multi-relational in nature.

Traversing relationship is fast as they are already captured into the DB, and there is no need to calculate them.

Graph base database mostly used for social networks, logistics, spatial data.

Neo4J, GraphDB, OrientDB, JanusGraph are some popular graph-based databases.

Graph Databases



Graph Databases







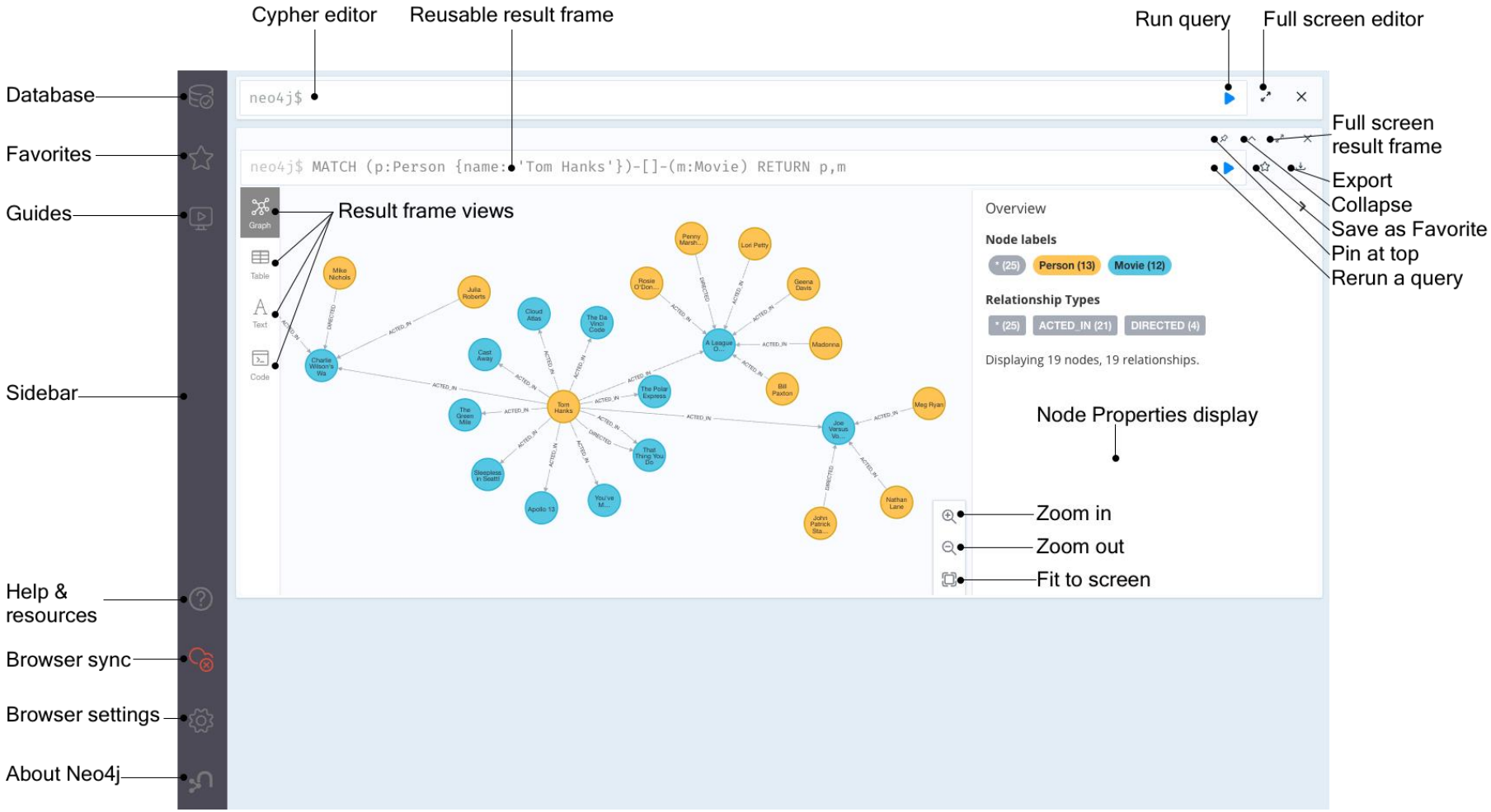


How Neo4j Works?

- Neo4j is a graph database that uses nodes, relationships, and properties to store and query data efficiently.
- Unlike relational databases (which use tables and joins), Neo4j is optimized for highly connected data and graph traversal operations.



Neo4j browser



https://neo4j.com/docs/browser-manual/current/visual-tour/

Key Concepts in Neo4j

📌 Nodes (Vertices)

- Represent **entities** (e.g., people, products, locations).
- Have **labels** to categorize them.
- Store **properties** (key-value pairs).

Example Node:

```
{
  "name": "Alice",
  "age": 30,
  "city": "New York"
}
```

Key Concepts in Neo4j

📌 Relationships (Edges)

- Represent **connections** between nodes.
- Have **direction** (-> or <-).
- Can have **properties** (e.g., date, strength).

Example Relationship:

(Alice) -[:FRIENDS_WITH {since: 2020}] -> (Bob)

Key Concepts in Neo4j

Properties

- Store additional information on **both nodes and relationships**.

Labels

- Categorize nodes (e.g., Person, Company, Movie).

Querying Neo4j with Cypher

Neo4j uses **Cypher**, a declarative query language designed for graph traversal.

Example: Creating a Node

CREATE (:Person {name: "Alice", age: 30, city: "New York"})

Example: Creating a Relationship

MATCH (a:Person {name: "Alice"}), (b:Person {name: "Bob"}) CREATE (a)-[:FRIENDS_WITH {since: 2020}]->(b)

Example: Finding Alice's Friends

MATCH (a:Person {name: "Alice"})-[:FRIENDS_WITH]->(friend) RETURN friend.name

Example: Finding Shortest Path

MATCH p = shortestPath((a:Person {name: "Alice"})-[:FRIENDS_WITH]-(b:Person {name: "Charlie"}))*

RETURN p

Querying Neo4j with Cypher

Example: Finding Shortest Path

```
MATCH p = shortestPath((a:Person {name: "Alice"})-[:FRIENDS_WITH*]-(b:Person {name: "Charlie"}))
RETURN p
```

Example Use Case

In a **social network graph**, this query helps:

- Find **the shortest connection** between two people.
- Determine **degrees of separation** (e.g., "Alice → Bob → Charlie" means Alice and Charlie are connected through Bob).
- Suggest **mutual connections** if they are indirectly linked.

CRUD operations In Neo4J using Cypher

Operation	Cypher Query
Create Node	`CREATE (:Person {name: "Alice"})`
Create Relationship	`CREATE (a)-[:FRIENDS_WITH]->(b)`
Read Node	`MATCH (p:Person) RETURN p`
Read Relationship	`MATCH (a)-[:FRIENDS_WITH]->(b) RETURN a, b`
Update Property	`SET p.age = 31`
Delete Node	`DELETE p`
Delete Relationship	`DELETE r`
Delete Node + Relationships	`DETACH DELETE p`

How Neo4j Works Internally

Storage Engine

- Stores data as **native graph structures** (not tables).
- Uses **adjacency lists** for fast traversal.
- Optimized for **index-free adjacency**, meaning relationships are stored **directly** with nodes.

Query Execution

- **Pattern Matching** – Finds subgraphs that match the query.
- **Indexing** – Uses B-tree indexes for node lookup.
- **Traversal Algorithms** – Uses BFS, DFS, and Dijkstra's algorithm for graph searches.

How Neo4j Works Internally

- **ACID Transactions**
  - **Atomic** (all-or-nothing updates).
  - **Consistent** (data integrity is maintained).
  - **Isolated** (parallel transactions don't interfere).
  - **Durable** (writes are persistent).

Why Use Neo4j?

- ✅ Best for Relationship-Heavy Data
- ✅ Fast Graph Traversals (No SQL Joins!)
- ✅ Flexible Schema (Evolves Dynamically)
- ✅ Scalable & Distributed (Neo4j Fabric for Large Graphs)

- ❌ Not Ideal for Transactional Systems (OLTP)
- ❌ Requires Learning Cypher (Different from SQL)

Real-World Use Cases

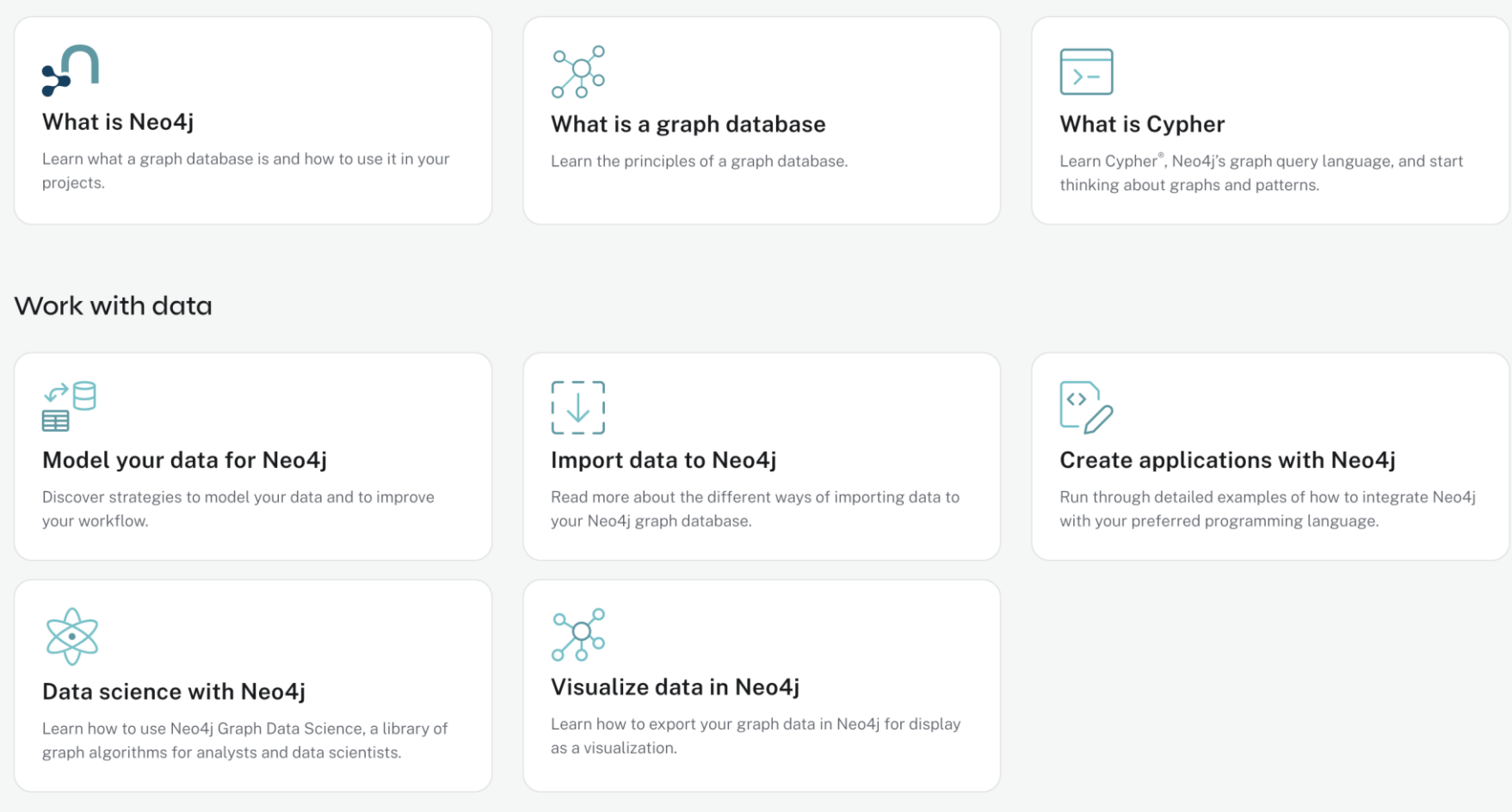
Neo4J (Network Exploration and Optimization 4 Java)

- **Social Networks** (LinkedIn, Facebook)
- **Recommendation Systems** (Netflix, Amazon)
- **Fraud Detection** (Financial Transactions)
- **Knowledge Graphs** (Google Knowledge Graph)
- **Supply Chain & Logistics** (Optimizing routes & dependencies)

Neo4j vs. SQL Databases

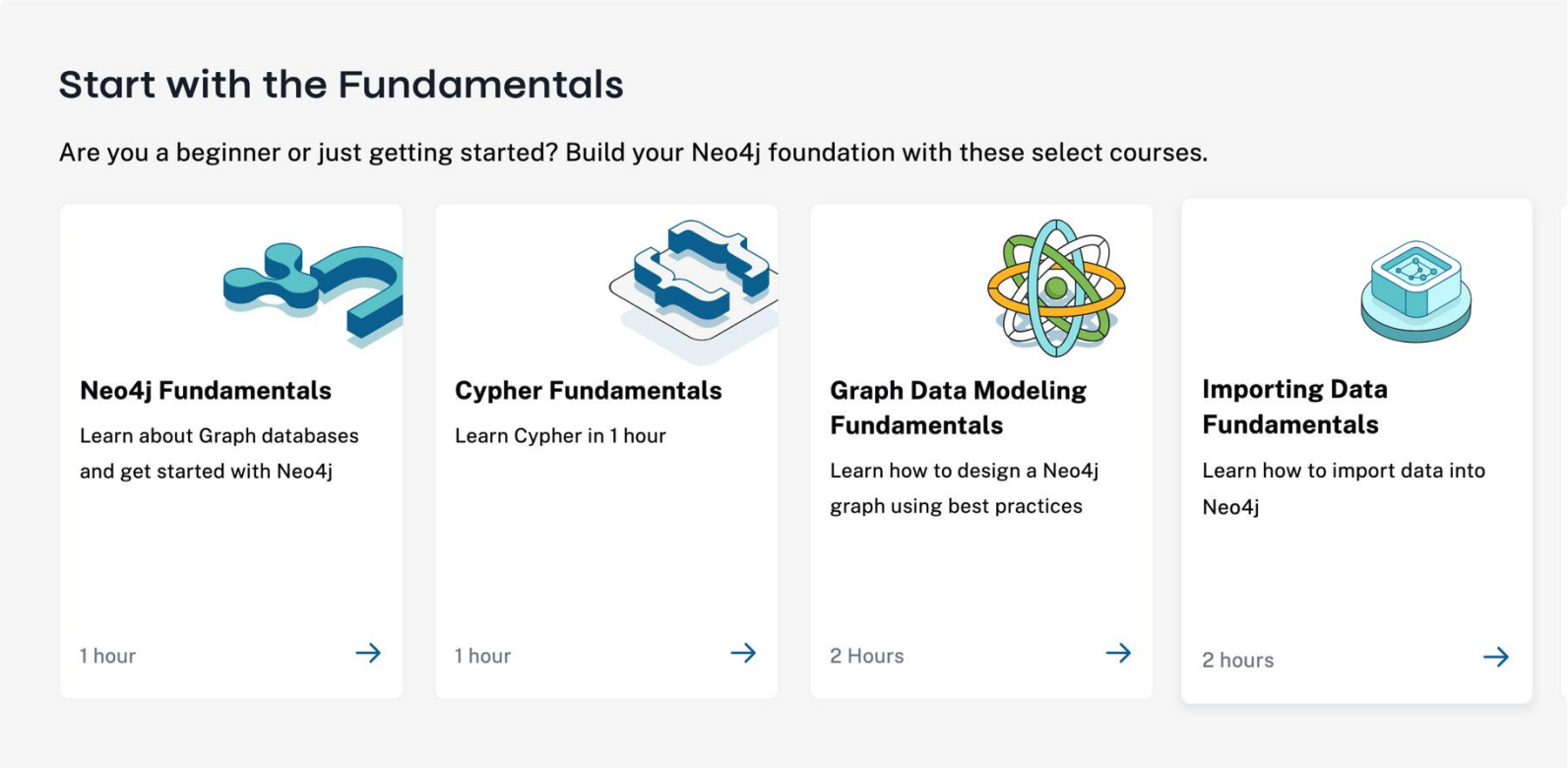
Feature	Neo4j (Graph DB)	SQL (Relational DB)
Data Model	Nodes & Relationships	Tables & Rows
Query Language	Cypher (Graph Traversal)	SQL (Joins & Aggregations)
Speed for Relationships	🚀 Fast (Index-free adjacency)	🐢 Slow (JOINs required)
Schema Flexibility	✅ Schema-less	❌ Fixed Schema
Best for	Complex relationships & networks	Structured tabular data

Get started with Neo4J



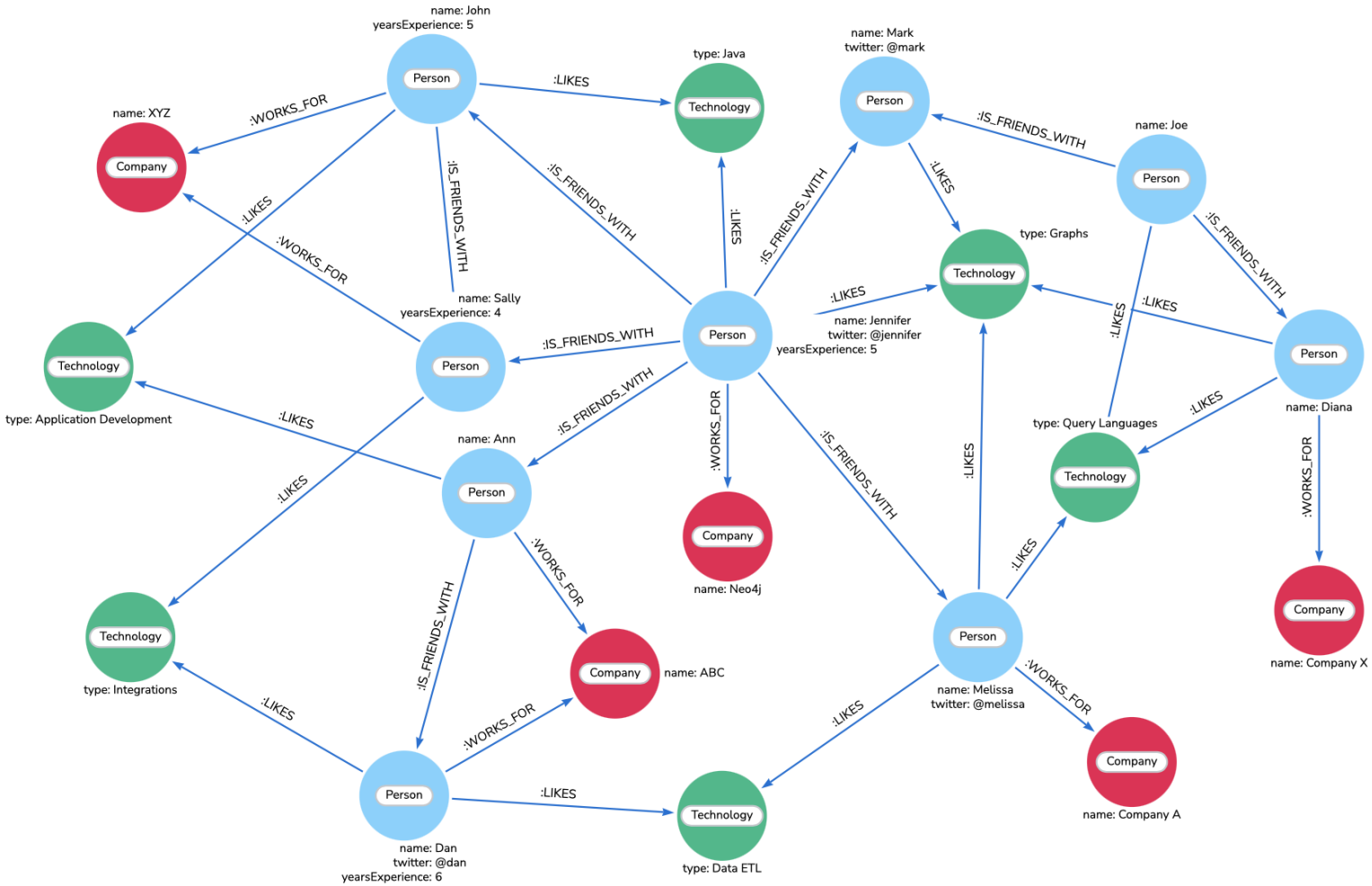
https://neo4j.com/docs/getting-started/

Neo4J – Graph Academy



https://graphacademy.neo4j.com/

Neo4J Example



Graph Databases: Use Cases

1 Facebook (Meta) – Social Network & Friend Connections

✅ Uses **Apache Giraph (Graph DB) & Neo4j** to:

- **Model friend relationships** (e.g., "Friends of Friends" suggestions).
- **Detect fake accounts & fraud patterns**.
- **Improve personalized news feed ranking**.

🔷 Why Graph DB?

A social network is a **natural graph** where:

nodes = people and **edges = friendships**.

Graph Databases: Use Cases

2 LinkedIn – Professional Network & Job Recommendations 🏢

✅ Uses **Apache TinkerPop & Neo4j** for:

- **Connection recommendations** ("People You May Know").
- **Job-to-candidate matching** (Graph-based skills analysis).
- **Influence & engagement analytics** (tracking interactions).

🔷 Why Graph DB?

It enables **fast traversal** of relationships (e.g., finding second-degree connections).

Graph Databases: Use Cases

3 PayPal – Fraud Detection & Risk Management 💳🔍

✅ Uses **Neo4j & ArangoDB** for:

- **Fraud detection** (detecting fraudulent transaction networks).
- **Customer relationship analytics** (understanding interactions).
- **Money laundering prevention** (tracing complex transactions).

🔷 Why Graph DB?

Traditional databases struggle with **deep relationship analysis** across many users.

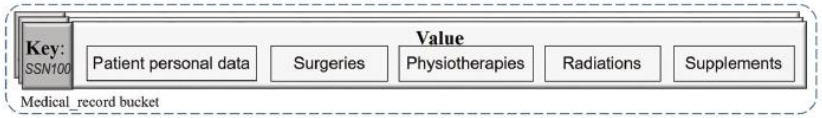
NoSQL stores

- Four major categories of NoSQL stores according to the supported data model.

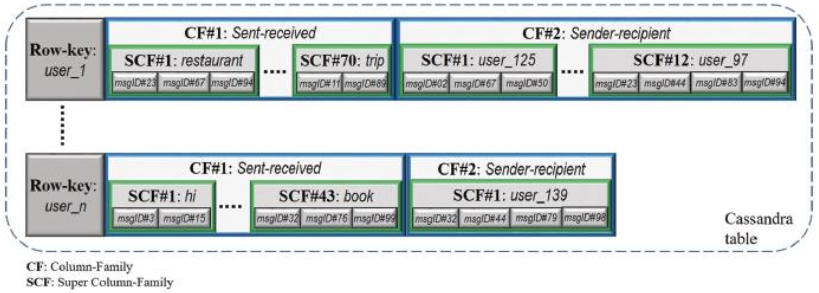
- The continuous development of major database technologies and some corresponding database systems.

- Source:
  - Ali Davoudian, Liu Chen, and Mengchi Liu. 2018. A Survey on NoSQL Stores. ACM Comput. Surv. 51, 2, Article 40 (June 2018), 43 pages
  - https://dl.acm.org/doi/10.1145/3158661

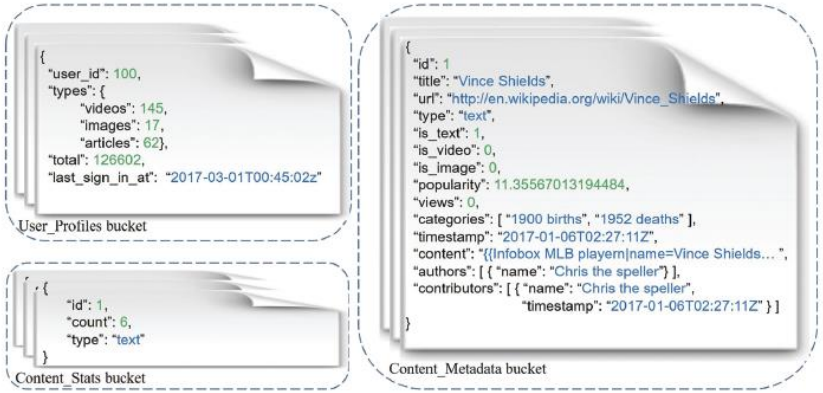
(a). Using a basic key-value data model for a health management system



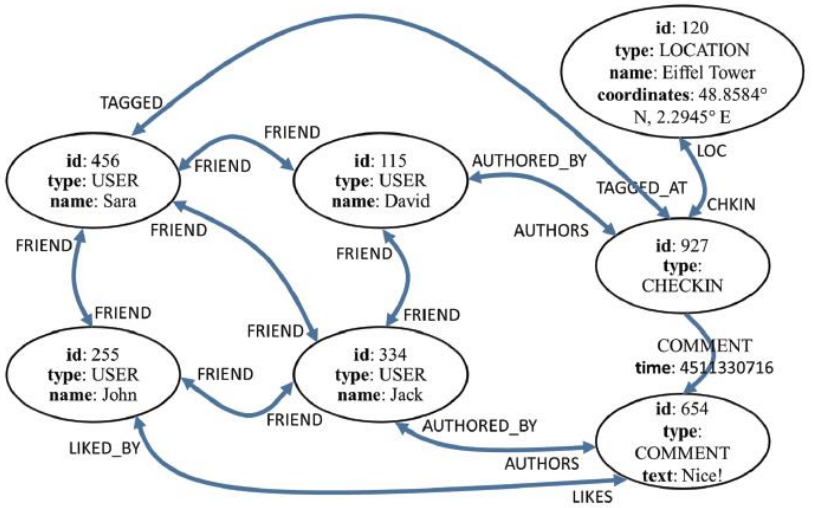
(b). Using the wide-column data model for the Facebook Inbox Search



(c). Using the document data model for the McGraw-Hill Education



(d). Facebook's social network uses the graph data model. As an illustration, suppose David along with his friend Sara visit the Eiffel Tower. David uses his cellphone to record this visit by 'checking in' to the Eiffel Tower and tagging Sara to let other friends know that she is also there. Jack writes a comment on this and John likes it.



Comparative view of key-value, column, document, and graph

- Selecting the right NoSQL database for our needs requires understanding the strengths and limitations of each type.

Data Organization:

Database Type	Organization
Key-value	Simple key-value pairs
Column	Data organized into columns (families) where each row has values only for existing columns
Document	JSON-like documents with various fields and values
Graph	Network of nodes (entities) connected by relationships (edges)

Comparative view of key-value, column, document, and graph

isec Engenharia

Schema:

Database Type	Schema
Key-value	Schema-less
Column	Flexible schema
Document	Schema-less or flexible schema
Graph	Flexible schema

Comparative view of key-value, column, document, and graph

isec Engenharia

Data Access:

Database Type	Schema
Key-value	Efficient retrieval by key, limited querying capabilities
Column	Efficient retrieval of specific columns across rows, limited row-level access
Document	Efficient retrieval of entire documents based on criteria, limited field-level access across documents
Graph	Efficient navigation based on relationships, querying nodes and relationships

Comparative view of key-value, column, document, and graph

isec Engenharia

Strengths:

Database Type	Schema
Key-value	Simple & fast, high performance, scalable
Column	Efficient column-level access, scalable
Document	Flexible data representation, agile development
Graph	Capturing complex relationships, fast relationship-based queries

Comparative view of key-value, column, document, and graph

Challenges:

Database Type	Schema
Key-value	Limited querying capabilities, data consistency considerations
Column	Complex querying for non-relational data, data consistency considerations
Document	Complex querying for joins and aggregations, data consistency considerations
Graph	Learning curve, limited querying for non-relational data, data consistency considerations

Comparative view of key-value, column, document, and graph

isec Engenharia

Use Cases:

Database Type	Schema
Key-value	Caching, session data, user profiles
Column	Sensor data, financial data, log analysis
Document	Content management, user profiles, product catalogs
Graph	Social networks, recommendation systems, knowledge graphs

Comparative view of key-value, column, document, and graph

Choosing the Right Database:

Consider your data model: Analyze the relationships between your data points. If these relationships are crucial, consider graph databases.

Querying needs: Assess the complexity of your queries. Key-value and column databases might not be ideal for elaborate joins or aggregations.

Performance and scalability: Evaluate your read/write requirements and potential data growth. All NoSQL options offer good scalability.

Data structure flexibility: Determine how rigid or adaptable your data structure needs to be. All NoSQL options offer more flexibility than relational databases.

Remember, understanding the specific needs and priorities is crucial for selecting the most appropriate NoSQL database type for our project.

Summary

- We move to implementation issues by looking at some example databases under the four key categories:
  - as an example of key-value databases we use Riak,
  - as an example, for document databases, we take MongoDB
  - to explore column-family databases we choose Cassandra,
  - and finally, Neo4J as an example of graph databases.
- The choice of examples not imply any recommendations.
- The aim is to give you a feel for the variety of stores that exist and for how different database technologies use the concepts outlined earlier.


Any
questions


NOSQL database. How do we query it??
write mapreduce filter
on sharded collection
How about going back to good old SQL?